Metrics

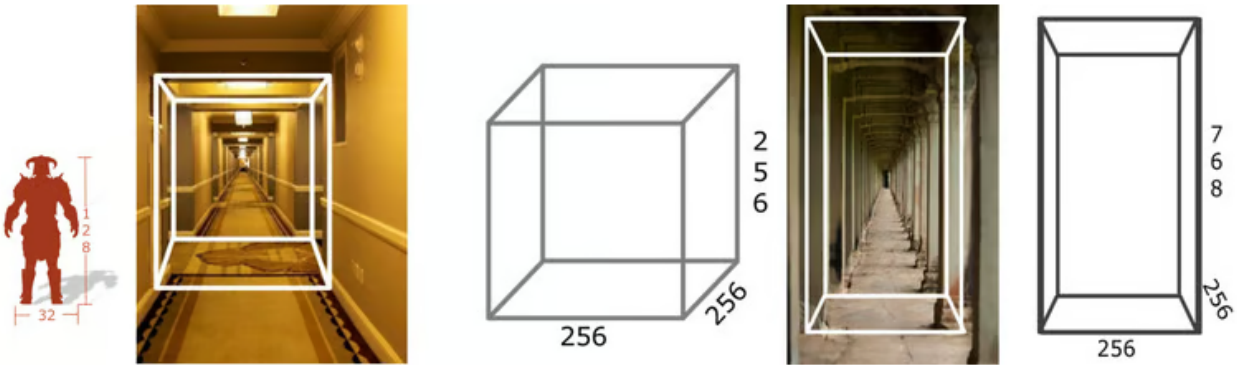
The proportions and distances of the level, how big the level feels

What are metrics?

In level design, **metrics** are the sense of scale, distance, and measurements across the entire level / game.

All level designers use at least two types of core metrics:

- **Player metrics** (or physics metrics) are factual numbers measured in-engine like player size, movement speed, maximum jump height, etc.
  - *describes* game physics, *derived* from the game engine
  - *example: "player size" is a specific fact that can be measured and verified*
- **Building metrics** are suggested floor / wall dimensions and guidelines that you codify for yourself based on your desired experience goals
  - *prescribes* game feel, *decided* by designer(s)
  - *example: "regular hallway size" is a guideline; what does "regular" mean for each project?*



comparing player size (physics) vs environment metrics, from "GDC 2013: Skyrim's Modular Approach to Level Design" by Joel Burgess and Nate Purkeypile (full talk and slides via JoelBurgess.com ↗)

Depending on the project, you might need *combat metrics* or *puzzle metrics* too.

But beware of over-reliance on metrics. Numbers and measurements may give the illusion of infallible design laws -- the illusion that following these numbers will give you the perfect level. You must reject this illusion! You cannot measure your way to a good game experience. Metrics are a helpful design tool to help you make decisions, but **metrics are not magic**.

> Note that "metric" is also a generic word for "measurement", it's not just a level design word. Across the game industry, we also have mobile game metrics ("KPI"), audience metrics, marketing metrics, monetization metrics... anything that can be measured.

Scale

Metrics help you establish **scale**, the general sense of how big the game world feels.

In level design we usually follow real world architecture built at a **human scale**↗**.** That means making sure rooms don't feel too small or too big for people. Proper scaling is very important when working in a realistic style, because it makes the level feel more plausible -- it will feel more like a real place built by humans and inhabited by humans.

But imagine a game about being a cat, it would need some rooms that feel too big (because cats are smaller than humans.) Or imagine a car game -- it would need houses that feel too small (because cars are bigger than humans).

For games we **"build at the player's scale"**, which may or may not be a human scale.

Video game scale is weird

The human scale is helpful, but video game spaces are not human. Video games often rely on an exaggerated sense of scale that does not correspond to any consistent real world measure.

In Doom, the player ("Doomguy") is 32 pixels wide, which translates to 1 meter wide. The shotgun is 63 pixels long, which is almost 2 meters (6 ft. 5 in.) long. Oh, and Doomguy runs at 30 units per frame, which is 40-50 MPH depending on which hotly debated scaling system ↗ you use. Either way, Doom scale doesn't make any sense ↗, but it feels nice to play anyway.

In The Elder Scrolls: Skyrim, the world area is about 14 square miles, the tallest mountain is 766.5 meters above sea level, and the player runs (not sprints) at 11.7 mph without tiring. That means the entire "province" of Skyrim is half the size of Manhattan, the tallest mountain is shorter than the Burj Khalifa in Dubai, and every peasant is a high performance marathon runner. By any real world measure, it is very small and weird. And yet, Skyrim somehow still feels like a large vast natural landscape with a huge alpine mountain, populated by typical people of average fitness.



the tallest mountain in Skyrim is more like the Matterhorn in Disneyland than any actual real world mountain

So when building video game spaces, remember that it should merely "feel" realistic as part of the mood, atmosphere, or aesthetic.

But if you approach level design too much like real world architecture, then ironically, your levels will feel too big, complicated, and implausible.

Don't follow real world scale.

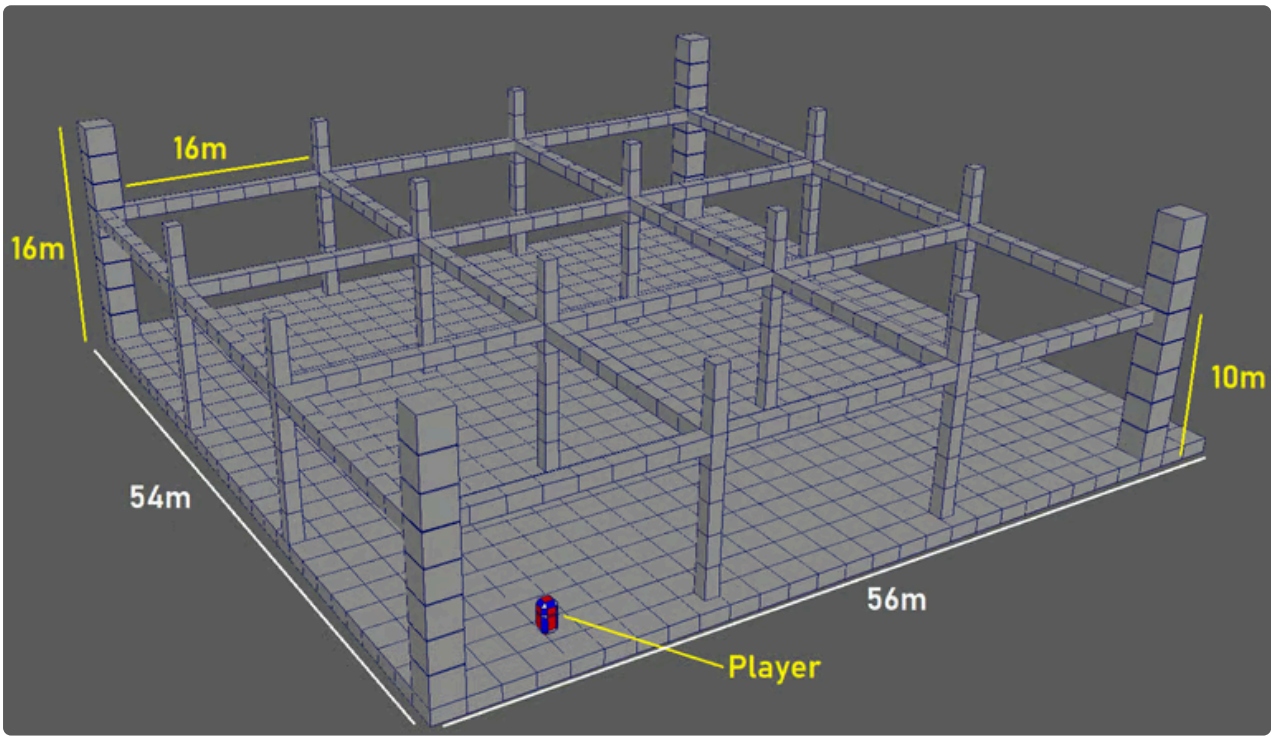
Scale advice

- "Big" levels feel big, but are actually much smaller than in the real world.
- Players regularly move at very fast speeds... that feel normal in the game world.

- The game world feels internally consistent, even if the specific numbers seem silly to think about.

Unfortunately it's not scientific. You'll only know "it feels right" when you're walking around inside your level and playtesting.

That's why we recommend considering metrics at the blockout phase. You can't really test metrics with a paper layout, but if you wait too late (like after an art pass) then it will be very time-consuming to make big changes. You might even have to redesign the entire level.



metrics for rooftop arena blockout by Ketul Majmudar for Spiderman (PS4) via blog post "Making Combat Spaces for SpidermanPS4" ↗

Basic metrics

In theory, you can build your game or level at whatever scale you want. In practice, physics implementations assume a default scale, and asset stores supply environment art at a common humanoid scale. Following convention is... convenient.

> For your convenience we provide common metrics for popular game engines below, but we encourage you to check out our full list of Tools.

Common player sizes

Game / Engine	Default units	Bounding box (width, height)	Eye height
Unity	Meters	1.0 x 1.8 m (or 1.0 x 2.0 m)	1.5 m - 1.7 m
Unreal	Centimeters	60 x 176 cm (half-height: 88)	152 cm (88+64)
Quake / Half-Life / Source Engine	Inches?*	32 x 72 in	64 in
(US, real-life)	Inches	20 x 69 in *(average adult)*	66 in

- **Bounding box / collision sizes are usually much bigger than the actual character model.** This bigger thicker size is for more stable movement. It's not a hitbox.
- Note that **in-game eye height is usually below where actual eyes would be**. Many FPS games put your eyes in your neck or chest. For more detail, see Metrics of perception - Camera height.
- ** Quake, Half-Life, Source Engine / "Quake lineage" games use "inches" lightly (eg. in Team Fortress 2, everyone is 7 feet tall↗) and don't really make sense.*

Common modern architecture dimensions

Game / Engine	Interior walls (depth, height)	Minimum hall width	Doors (width, height)	Steps (height, depth)
Unity	0.1 x 3.0 m	2.0 m	1.25 x 2.5 m	0.1 x 0.15m
Unreal	20 x 300 cm	150 cm	110 x 220 cm	15 x 25 cm
Quake / Half-Life / Source Engine	16 x 128 in	64 in	56 x 112 in	8 x 12 in
(US, real-life)	6 x 96 in	48 in	36 x 80 in	7 x 11 in ("7-11")

- The minimum hallway width should be at least double the player width. Even then, it will feel a bit narrow and uncomfortable.
- Modern doors should be narrower than hallways to allow space for a door frame.
- Modern flights of stairs have landings / platforms every 12-16 steps. Long or tall stairs will feel industrial, monumental, or otherwise non-domestic.
- Modern stairs should follow a 30-35 degree slope. `arctan(7/11) = 32 degrees`
- In Source 1 and older engines, level grids followed power-of-two numbers. Unity users often prefer 1.0's and 10's. Unreal users are split based on their age. Use whatever number roundings that you and your collaborators prefer.
- When building in a realistic modern style, do some research beforehand: gather plans, blueprints, schematics, or even read through local building codes.



building a deck that conforms to building codes by CorporateM (via Wikipedia, CC-BY-SA 4.0 Intl license) ↗

Deviating from conventional metrics

Deviate meaningfully from typical metrics, proportions, and spaces.

Steep stairs will feel more harrowing and awkward than shallow stairs, and if that's the experience goal you want, then go for it. Often the best part of a level is the uncommon and unexpected. But to make something feel unusual, you must contrast it with enough usual common elements first.

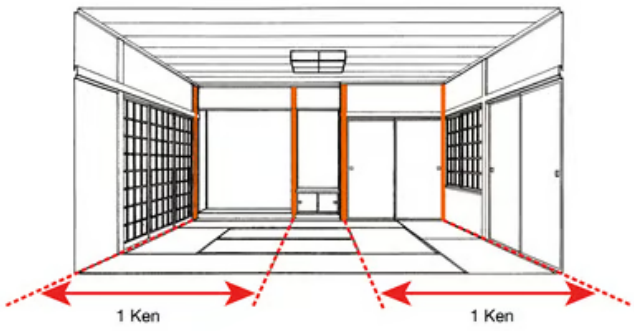
What if the level is stylized / isn't realistic?

Even with stylized non-realistic art styles, metrics still need to feel **internally consistent** to the game world / design norms. For example, the steps in Quake 1 maps are often 16 inches tall, but the rest of the game world is appropriately big and chunky to match that proportion.

What if the level isn't modern?

Do research on the type of architecture you're using. For example, traditional Japanese architecture uses the *ken* (間) ↗ proportional system. Italian Renaissance architects used the golden ratio (1 : 1.618) in the vein of Ancient Greek and Roman architecture, etc.

> *For more information on planning, see Pre-production and Research.*



Meter (m)	Sun	Shaku	Ken
0.0303	1	0.1	0.0166666
0.30303	10	1	0.166666
1	33	3.3	0.55
1.81818	60	6	1

diagram and chart with "ken" measurement system used in a traditional Japanese room (via Musashino Art University glossary ↗)

How to use metrics

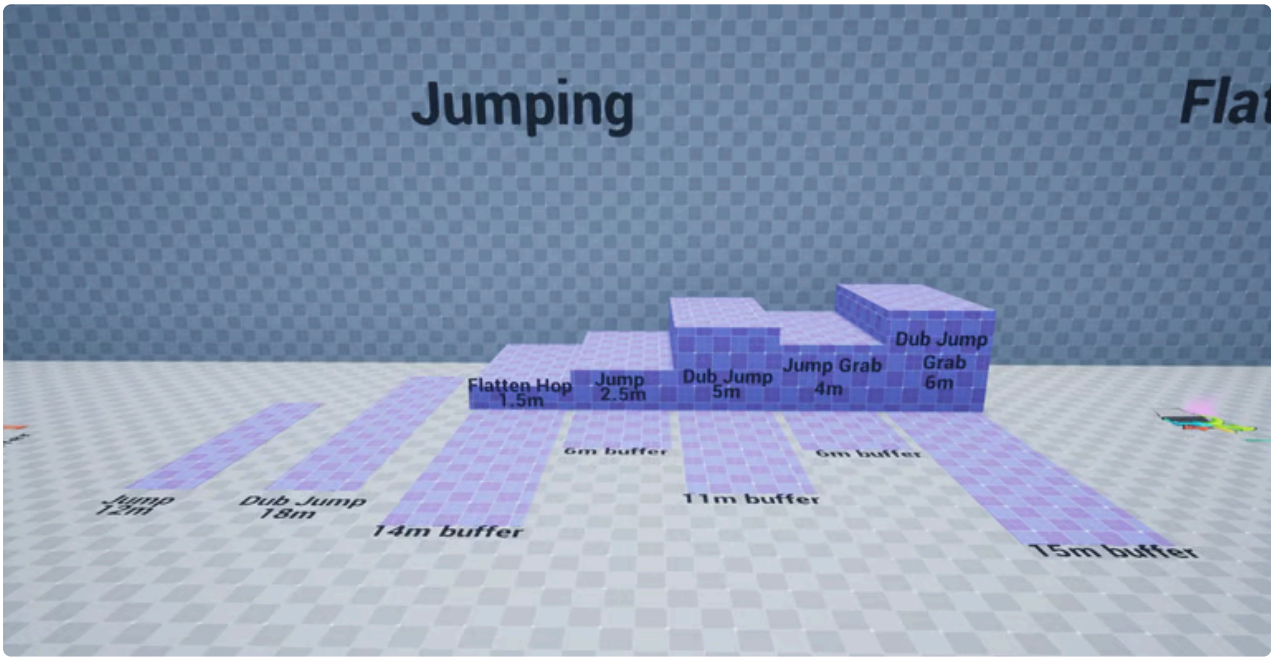
Some ways to bring metrics into your level design process:

- **Build a metrics zoo test map**
  - Prototype core player metrics
  - Use world-aligned grid textures
  - Test modular kit
- **Metrics debug tools, editor previews**
- **Codify shared building metrics**

Build a metrics zoo test map

Build a "metrics zoo" / "metrics playground" developer-only test map that exhibits the different gameplay objects, modules, and prefabs across all your levels.

1. **Blockout.** Make a simple level with various rooms, hallways, and floor heights.
2. **Playtest for usability.** Walk around with gravity, jump on objects, be a general nuisance.
3. **Playtest for feel.** Pay special attention to the sizes of walls, doors, and floors. Does the height of the stair steps feel right? Do doors feel too narrow or too short?
4. **Iterate.** After each metrics revision, playtest and walk through again. Repeat.

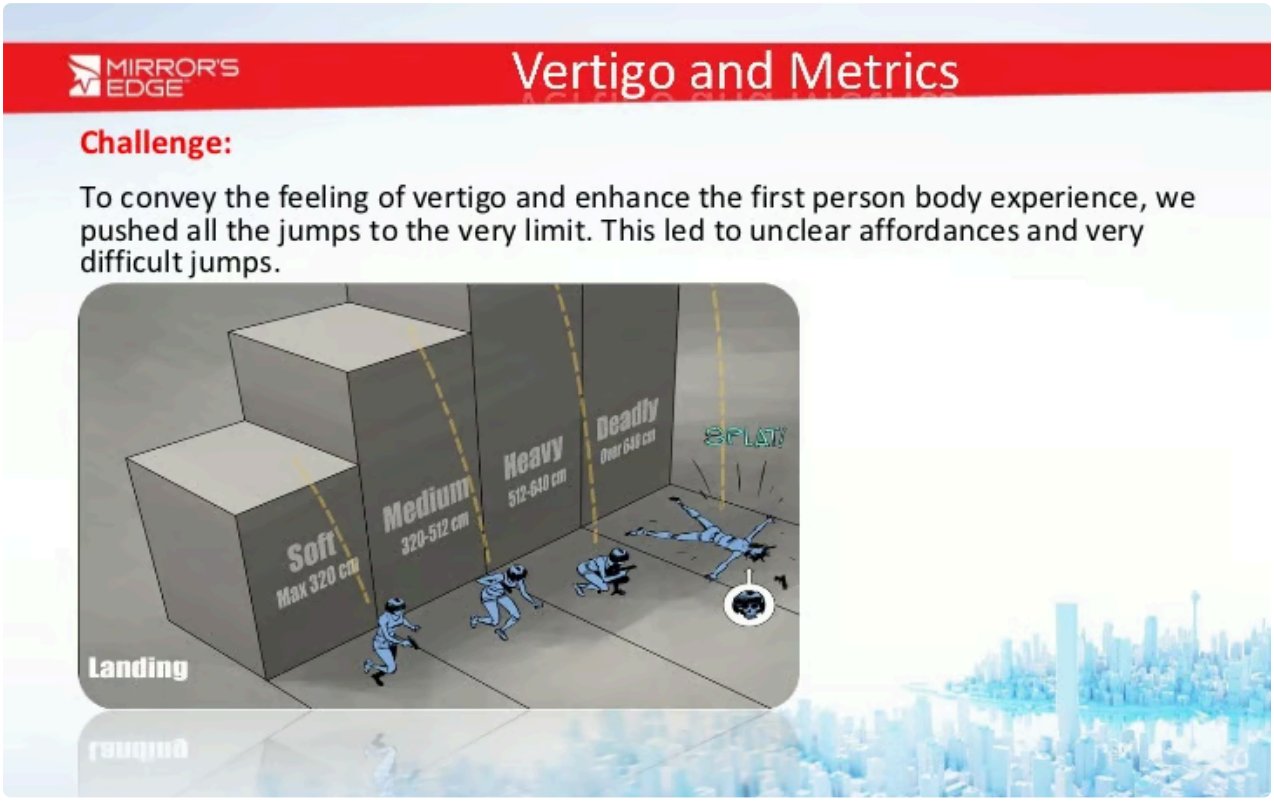


example metrics map with measurements by Tyler McCombs (via Twitter account @TylerDanger ↗)

Some example core player physics metrics

- **Basic movement**
  - Walk speed, run speed
  - Maximum step height, maximum angle slope incline
  - Minimum hallway width, minimum ceiling height
- **Jump, fall, drop, height**
  - Maximum jump height, maximum jump distance, running jump distance

- Gravity strength, fall speed
- Fall damage height threshold for 1% damage vs 100% damage lethal fall



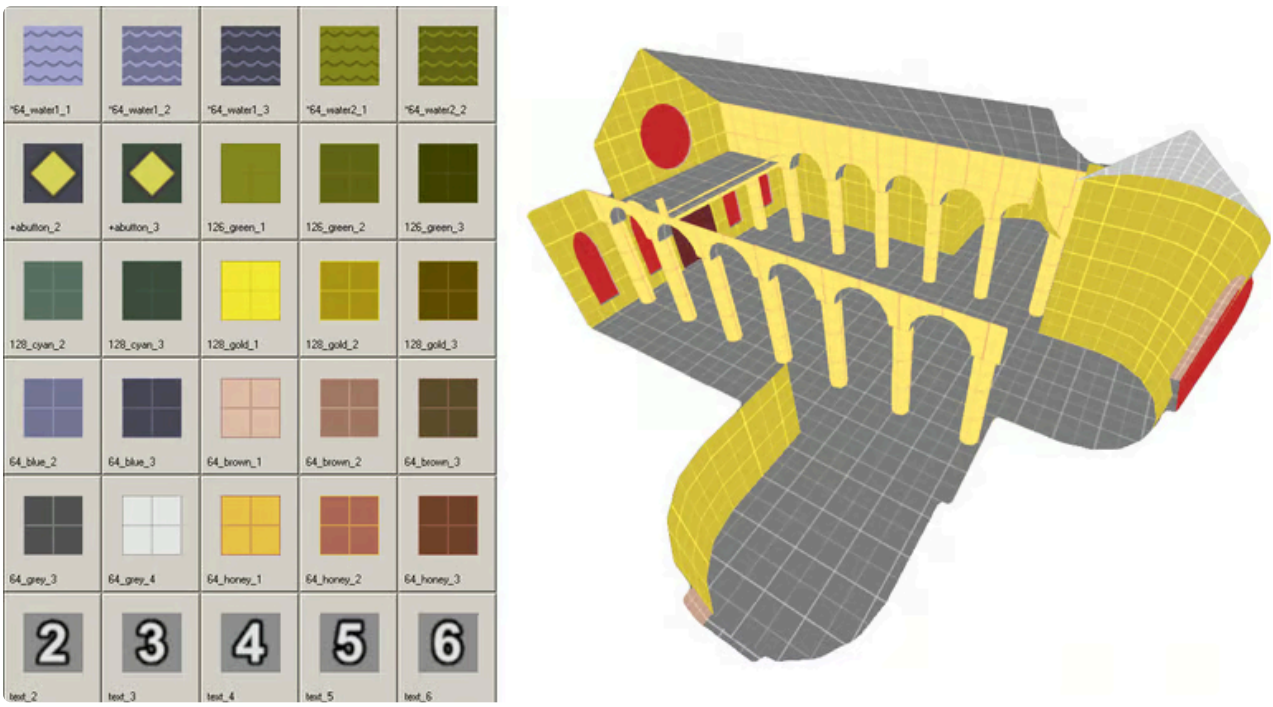
internal fall damage metrics diagram for Mirror's Edge, from "Level Design Challenges & Solutions - Mirror's Edge" by Elisabetta Silli (via Slideshare) ↗

Use world-aligned grid textures

When blocking out, use some kind of world texture with a little surface detail.

- Do not use flat solid colors, these lose visual scale and depth cues while prototyping.
- Do not use detailed textures, these are visually busy, can distort scale, and are distracting.

We recommend using placeholder grid / checkerboard textures, anything with repeating lines at regular intervals that will help you eyeball distances and proportions as you build your blockout. Check our **Resources** for links to various prototyping texture packs.

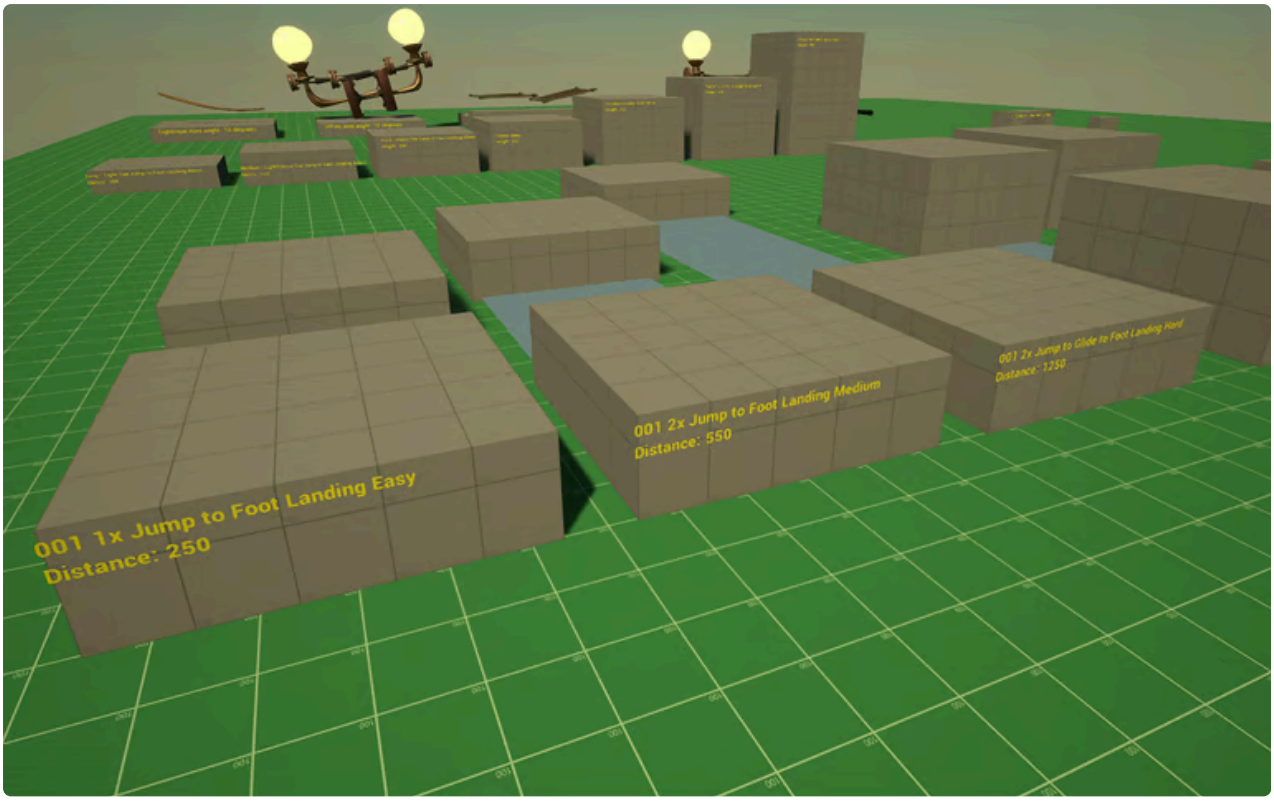


gridded prototyping textures (Prototype.wad) and "church" blockout by Khreathor (via artist's website and download page ↗)

While building, make sure the texture scale stays constant and independent of the 3D object scale.

To ensure a constant scale in your game engine:

- **Quake / Half-Life / Source:** make sure your level editor's **"texture lock" is disabled**, and set all brush faces to use world axis alignment. For TrenchBroom, see "TrenchBroom Manual: Working With Textures" ↗.
- **Any modern game engine:** configure the shader or material to use a **triplanar projection** with a tilting grid texture. This has the added benefit of automating UV unwraps across all objects, so you can focus more on geometry rather than texture alignment.
  - **Unity:** visual shader editor has a built-in Triplanar Node ↗, or to go deep on coding your own shader see Ben Golus' post "Normal Mapping for a Triplanar Shader" ↗
  - **Unreal:** material editor has a built-in WorldAlignedTexture Node ↗, or to go deep into coding your own shader see Ryan DowlingSoka's post "Triplanar, Dithered Triplanar, and Biplanar Mapping in Unreal" ↗



screenshot of internal metrics test level for Psychonauts 2; note the gridded prototyping textures! (via "Behind the Code: Designing Raz's Jump" by Devin Kelly-Sneed ↗)

Test modular kit metrics

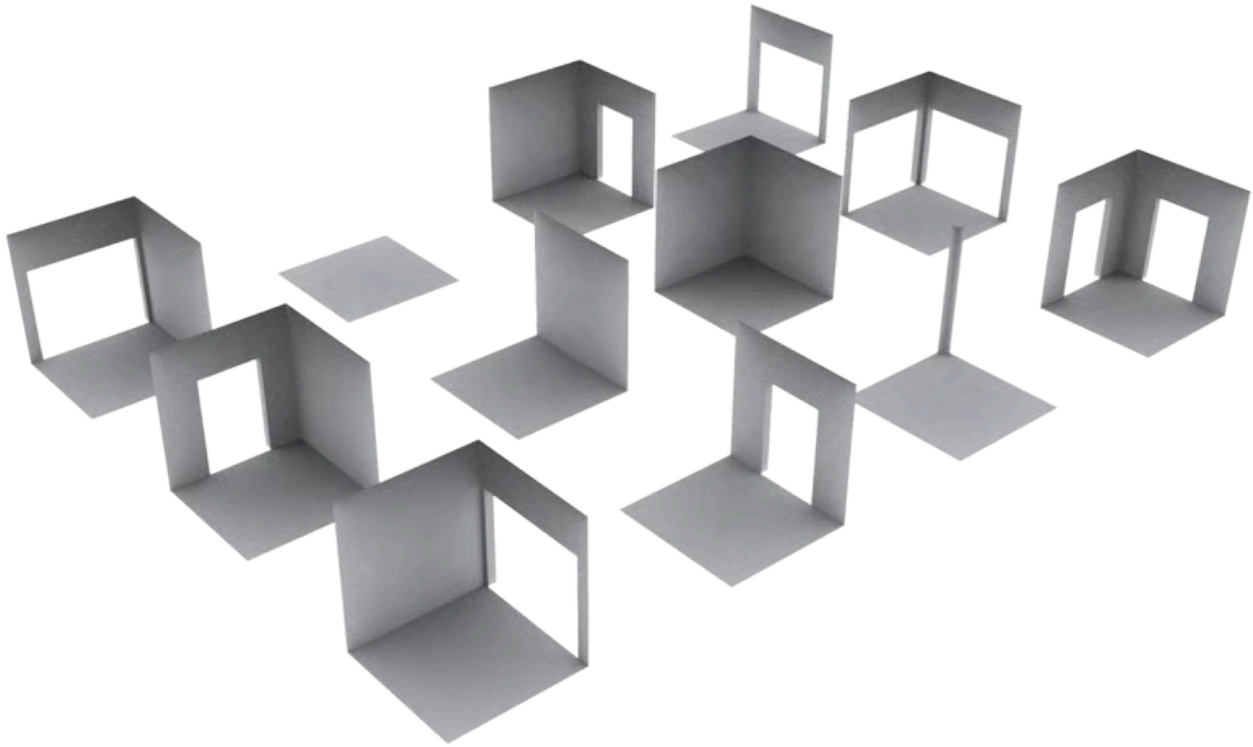
With a **modular kit**, you build a level with pre-made tiles and components that snap together.

What's the best size for a doorway? You don't have to worry anymore; that metric is baked into the doorway module. In modular construction, you measure once, and basically copy-and-paste that measurement across the entire level.

However, it is common to iterate on a modular kit on the fly -- adjusting tile sizes and measurements, refining and art passing modules later, etc. If you make big changes to the modular kit's design assumptions, such as tile grid size or granularity, then you will likely have to rebuild everything because the new kit won't work for levels that used the old kit.

The best way to test the metrics and scale of a modular kit is to use it! Blockout with it, try to build hallways that loop back on themselves, then playtest and walk around in it.

> *For about modular construction, see Modular kit design.*



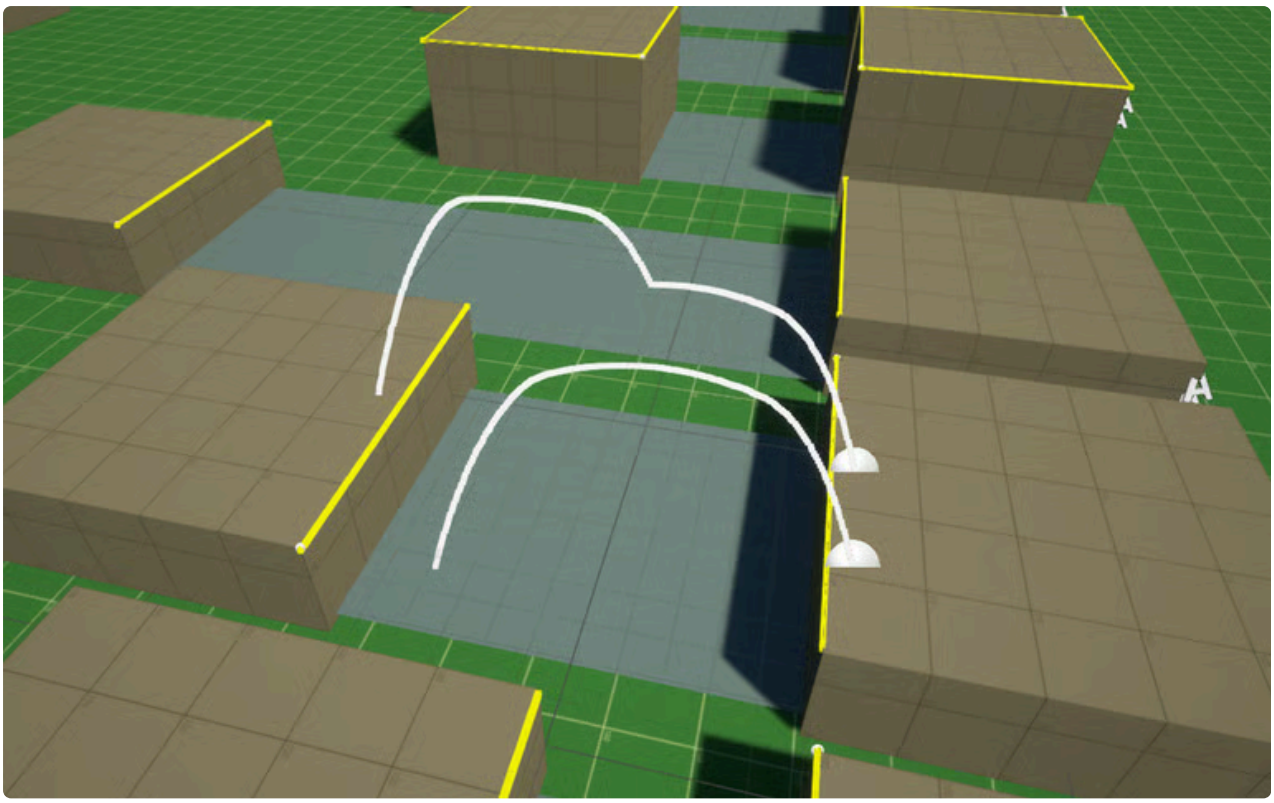
3D tiles ("modules") with pre-measured metrics used for blockouts for Skyrim, image by Joel Burgess (via JoelBurgess.com ↗)

Metrics debug tools and guides

For action games with a heavy focus on player movement and platforming, it can be difficult to predict whether a wall is too high to jump, or a gap too wide, etc. An in-editor metrics measurement tool / visualization / trajectory preview can help resolve these ambiguities.

For Psychonauts 2, developers implemented a "jump preview" editor visualization so that level designers could easily predict a player's jump arc:

> *"The jump metrics tool is designed to be placed into the level when the game is not running to visualize Raz's jump arc. This can be used to help place objects along an action path so they'll be reachable by the intended jump type. The tool runs the jump simulation code at edit time, while the game is not running. Any changes to the jump tuning would be reflected by the preview arc."*
>
> -- Devin Kelly-Sneed, "Behind the Code: Designing Raz's Jump" ↗



screenshot of "jump preview" editor visualization, showing a double jump and a single jump arc in a metrics test level from Psychonauts 2; image by Devin Kelly-Sneed (via DoubleFine.com: "Behind the Code: Designing Raz's Jump" ↗)

Codify shared building metrics

Beyond player metrics like maximum movement speed or jump height, you can also set recommended building metrics for yourself as a guideline -- and these shared guidelines across a design team can help make construction and collaboration easier. Some examples:

Combat metrics

- What is "close range" or "long range"? How big is a "big fight"?
- How tall is cover, how high should a window be?

Pacing metrics

- How long should each map or chapter be altogether? How far should the exit be, how much backtracking should be allowed?
- How far apart should the various map objectives be?

- How many resources or rewards can be placed in each stretch of the level?

Storytelling metrics

- How long is this NPC dialogue, and how far can the player travel while they are talking?
- How many narrative assets such as audio logs, readables, codex collectibles, etc. should be placed in a given part of the level?

Optimization metrics

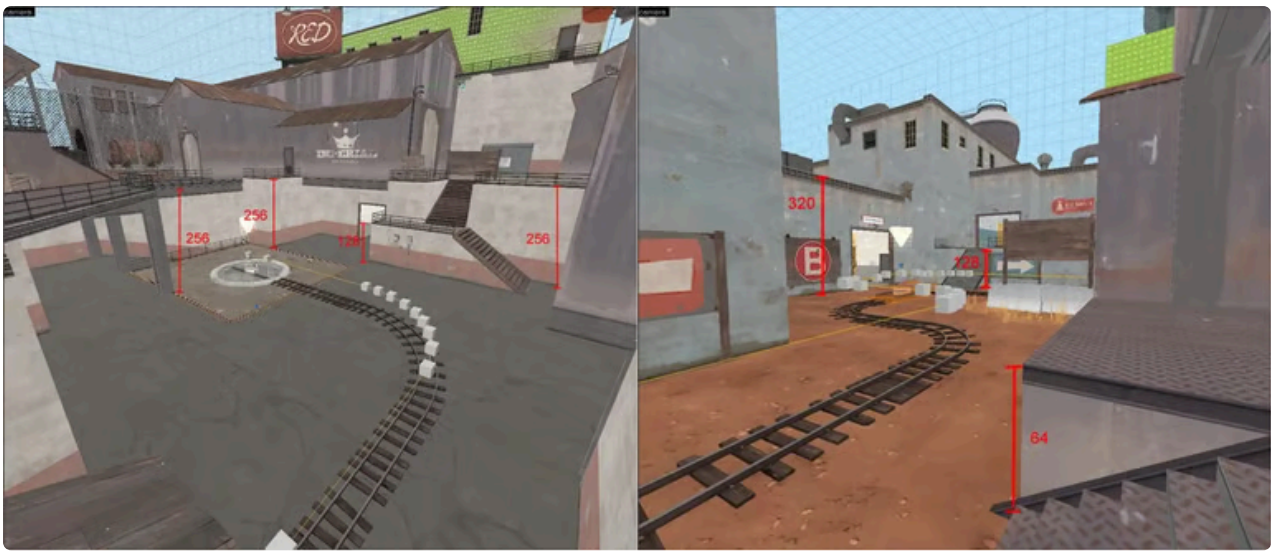
- If we are streaming in chunks of level data, how big should each chunk be?
- How long does the level transition hallway or sequence need to be, to mask the level streaming time?
- How many different meshes, materials, or shaders, can we load for each level?

Some example combat metrics for Team Fortress 2, based on observation and analysis from the TF2maps community ↗:

- *Close range weapons: 256 units or less*
- *Medium range weapons: 1024 or less (and projectiles become easier to dodge, etc)*
- *Rocket spam and snipers: 2048*
- *Maximum drop height without fall damage: 256*

These metrics are reflected in the arena design of Team Fortress 2 maps like Badwater Basin below. Most drops are 256 units high, with maybe a few drops at 320 to apply a small penalty for any falling players. However there aren't many 512 unit drops because that would limit player options too harshly, and also render close range weapons much less effective.

> *For examples of detailed metrics, see Doom metrics and Quake metrics.*



editor screenshots of Badwater Basin (Team Fortress 2) with common height measurements; most walls are 128 or 256 units high, while walkable platforms without railings are 64 or 128 units high... by Grazr (via tf2maps.net forum thread: "[GUIDE] Scale and your map.") ↗

Metrics of perception

What affects the player's perception of size and speed? Basically anything on the screen.

Camera height

> *"Some animals are lightning fast. Others are pitifully slow. But slow and fast are relative terms. Four miles per hour doesn't feel very fast to a human: it's approximately one body-length per second. But to a small insect it's approximately 100 body-lengths per second. A human traveling that fast would be going 20,000 miles per hour! This is why some animals' brains process visual stimuli much faster than ours, and why they have better reflexes (think about how hard it is to swat a fly). What does it feel like to comprehend the world at such speeds?"*
>
> -- Alex Reisner, creator of SpeedOfAnimals.com ↗

If you are prototyping a game, finalize the camera height / eye offset as soon as possible, because it will affect how big the entire game world and all its characters feel. The camera height is one of the key variables that affect the player's sense of a virtual body, because "slow" and "fast" are relative to body size.

For suggested eye heights, see the Basic metrics section above.

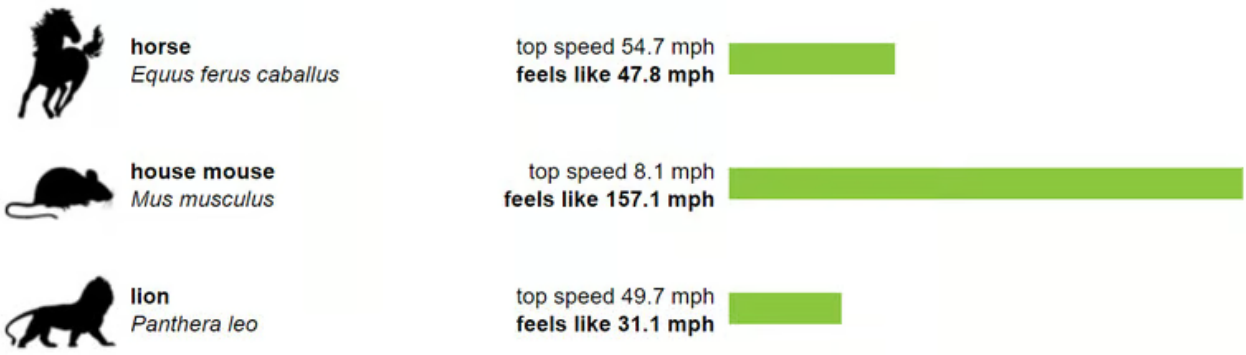


diagram showing how a mouse experiences the world at the average US adult human equivalent of 157.1 mph, by Alex Reisner (via SpeedOfAnimals.com ↗)

Local reference points, composition, texturing, fog

TODO: more local context, objects along the line of movement

Camera field of view (FOV)

Your sense of perceived speed depends on how much you are seeing and the rate of visual change within your vision. A high field of view ↗ (FOV) will zoom-out to produce fisheye distortion that increases apparent speeds at the edge of the screen (periphery).

Note that there are three ways to measure field of view -- horizontal, vertical, and diagonal -- but for games we generally refer to the horizontal FOV so that it scales well with widescreen setups.

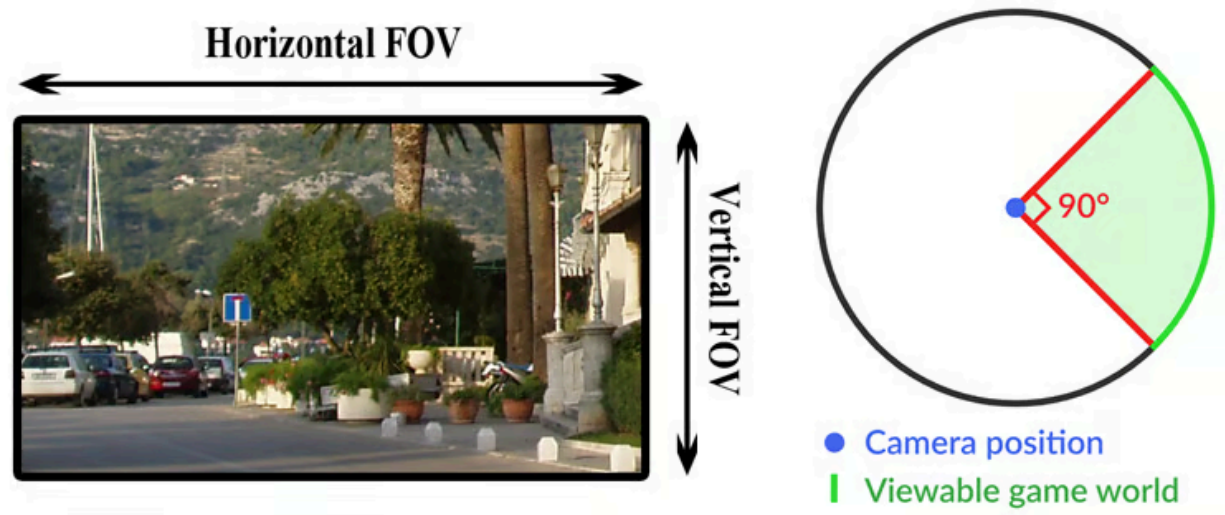
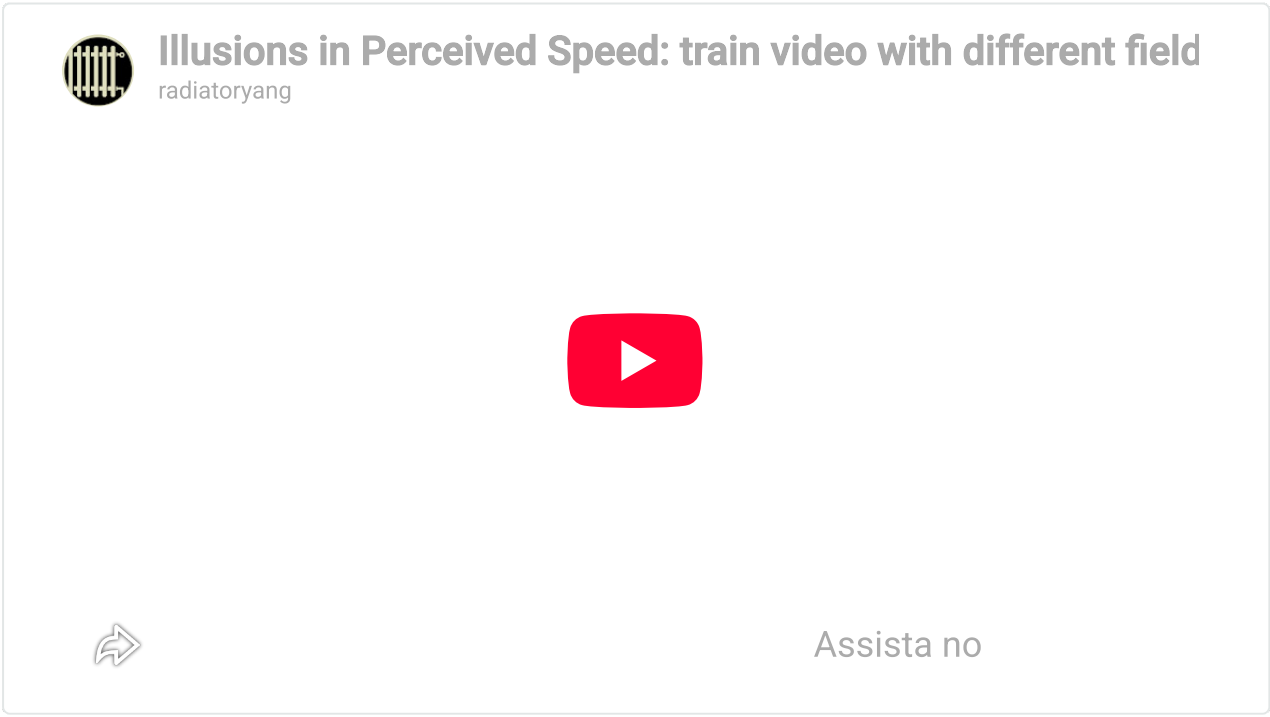


diagram illustrating field of view (FOV), image by Jelosantisterio via Wikimedia.org (under CC-BY-SA 3.0 license) ↗

Some players prefer high FOVs because they can see more of the world around them and gain better situational awareness, and there have also been studies that suggest a high FOV helps some players mitigate motion sickness while playing first person games.

On top of the base FOV setting, many third person action games shift to a higher FOV when the player sprints or accelerates. This FOV change gives a greater sensation of speed, even if the actual speed increase is negligible. In contrast, some virtual reality games purposely occlude and mask peripheral vision to decrease this sense of acceleration and mitigate VR sickness ↗.



video from front of a train with different fields of view that affect the perceived speed; by Akiyoshi Kitaoka ↗

The default Quake (1996) field of view was 90 degrees, and competitive players often set their camera to 120 degrees so that they could see more of their surroundings. This made more sense when CRT monitors and TVs had a 4:3 aspect ratio, but feels different on a modern 16:9 widescreen display.

Today, most commercial 3D action games tend to offer accessibility settings with an FOV range slider for users to adjust the camera to their personal comfort, usually between 60 and 90 degrees, but default to a narrower FOV. A high FOV distorts the image in an unrealistic / unflattering way. Meanwhile, a lower FOV zooms-in and "flattens" the view depth, giving a greater sense of closeness and intimacy with the game world. If your game is about looking up-close at characters' faces, you would err on a narrower FOV so that the faces don't look strange or distant.

Portrait photographers often try different lens focal lengths to get the most flattering distortion and closeness with their subject. 20 mm (84° FOV) narrows the face with extreme proximity, 70-100 mm (29-20° FOV) is more naturalistic, and a long 200 mm (10° FOV) telephoto lens widens and flattens the face with a longer distance from the subject.



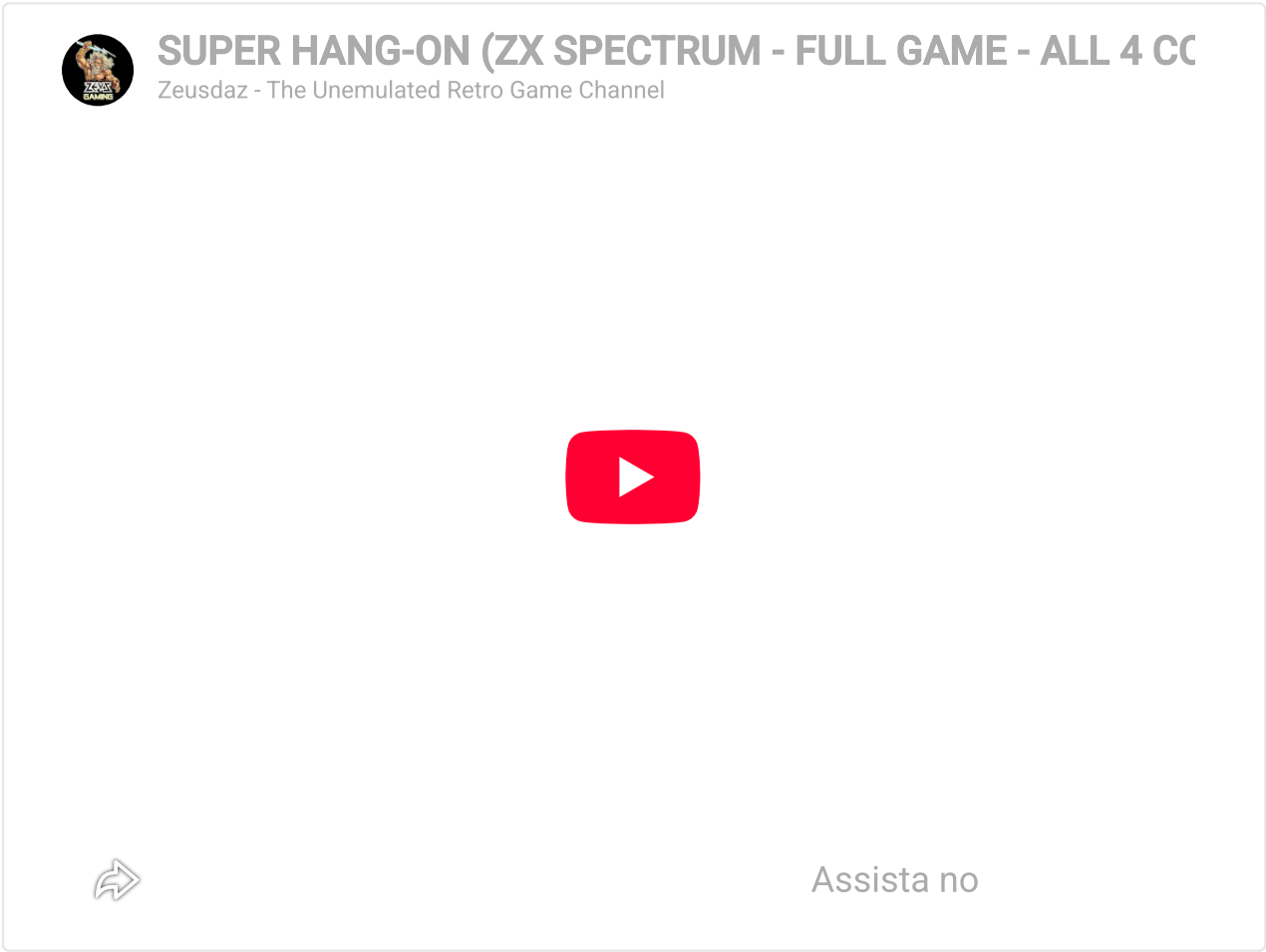
animated GIF of portraits with different focal lengths by Dan Vojtech (via DanVojtech.cz) ↗

Sound design, HUD / UI, and NPC AI as speed metrics

The brain processes all senses together. Vision isn't just a matter of seeing, it is also a matter of sound, memory, knowledge, etc.

To demonstrate, there is a well-known experiment in game feel ↗ involving two identical circles sliding past each other to swap places. Without any sound, the circles will appear to be just exchanging positions, but if you play a collision sound when the circles intersect, then the circles will appear to collide and bounce off each other instead.

For a more direct example of how other stimuli affect our sense of speed, consider the ZX Spectrum port of Super Hang-On (1987). In this single player motorcycle racing game, the "turbo" button temporarily increases the player's max speed limit to 324 km/h, beyond the normal limit of 280 km/h. However, the game does not render faster nor does the player actually move faster; instead, the engine sound gets louder, numbers get bigger, and opponent NPCs slow down. These contextual cues combine to give a faster feel without actually adjusting the camera nor the player.



footage of ZX Spectrum port of Super Hang-On (1987) with use of "turbo" ability at 0:39

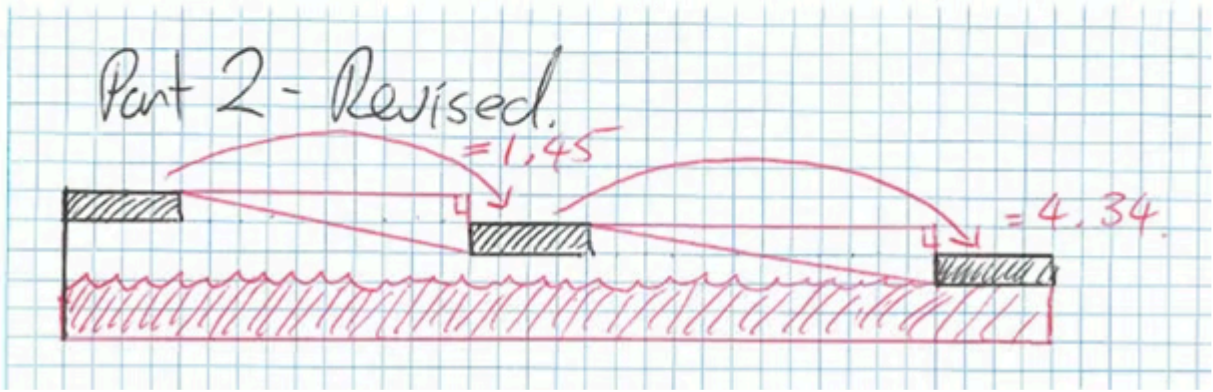
The church of metrics: "rational game design"

A game industry practice known as **rational game design (RGD) / rational level design (RLD)** focuses on measuring metrics to plot parameterized game mechanics against game difficulty in a coordinate space, with the goal of mapping out a game's possibility space in a quantified way.

Basically, give everything in the game a number, and then add up all the numbers to help guide your design decisions. If you don't like how the numbers add up, change them. This general workflow looks like:

1. Design game mechanics, and identify various parameters and modifiers.
2. Plan levels across the entire game with different combinations of mechanics and modifiers.
3. Quantify parameters for each mechanic (e.g. 0% difficulty jump is a 3 m wide gap between platforms, while a 100% difficulty jump is 10 m wide, etc.)
4. Iterate on each level's metrics to conform to desired difficulty scores and target values.

	Jumping Puzzle 1		
	Part 1	Part 2	Part 3
Easy Jump	3	0	2
Medium Jump	0	1	0
Hard Jump	0	1	1
Jump Difficulty	3	8	8
Modified By Steps	3.15	8.08	8.4
Modified By Verticality	3.15	5.85	8.4



RLD spreadsheet and diagram from "The Rational Design Handbook: An Intro to RLD" by Luke McMillan ↗

Criticism of rational game design

Supporters claim this design method represents a more "rational" and "scientific" approach to plotting and pacing the player experience. Unfortunately there are very few public resources on how to practice RGD, which is convenient because that means we can't actually evaluate its methodology or claims -- its evangelists can always claim that we haven't read the proper holy scriptures, and so we don't truly understand how to practice it. What little documentation here is cobbled together from the few articles and resources available online, with much of the material guarded as a trade secret by Ubisoft.

We caution against any uncritical belief in any single design method, especially something that touts supposed objectivity / ahistorical understandings of "form follows function." And we remain skeptical of any method that claims to capture conceptual depth with a number. The more technical the design formula, the more formulaic the design will be -- a common critique leveled at many Ubisoft games.

But if RGD / RLD serves as a useful design tool that yields good results for the type of game you're making, then there's no harm in using it with *moderation*. Collecting data, measuring spaces and distances, and playtesting are all certainly good design practices. Just don't drink too much of this *rationality Kool-Aid.*



RGD variety matrix for Rayman Legends with two possible level arrangements at the bottom, by Chris McEntee ↗

To review...

Metrics are useful measurements to help blockout levels at a consistent scale. If a level feels weird or awkward to play, it may need better attention to metrics.

To recreate a real world place, don't copy the exact measurements -- video game scale is often weird and inconsistent. Instead, **scale depends more on your intuition of "what feels right."**

To use metrics, **blockout a metrics test map / zoo / playground** with simple gridded textures, and then **walk around in it**. If you're using a modular kit, then build a test level and walk around in it, to make sure the modules are at the right scale and dimensions you need.

The most important level design metrics to consider are usually **player physics metrics** like movement speed, stair step height, and jump distance. You'll also probably want common **building metrics** like typical doorway width, window height, wall height, etc.

Metrics reflect human perception, which is affected by anything on screen or even game audio.

Many metrics are just "good numbers" and guidelines you give yourself, they're not a scientific infallible way to perfect level design. **Metrics are useful, but metrics are not magic.**

Now what?

- Use metrics to build a blockout.

Further reading on metrics

- Examples of detailed metrics breakdowns:
  - Doom metrics
  - Quake metrics

- TF2Maps.net: "Scale and your map"↗ guide by Grazr. Even if you're not making a Team Fortress 2 map, it is useful to see how they derive their suggested metrics here.
- Source Engine / Half-Life 2 metrics (Valve Developer wiki)↗

Modular kit design

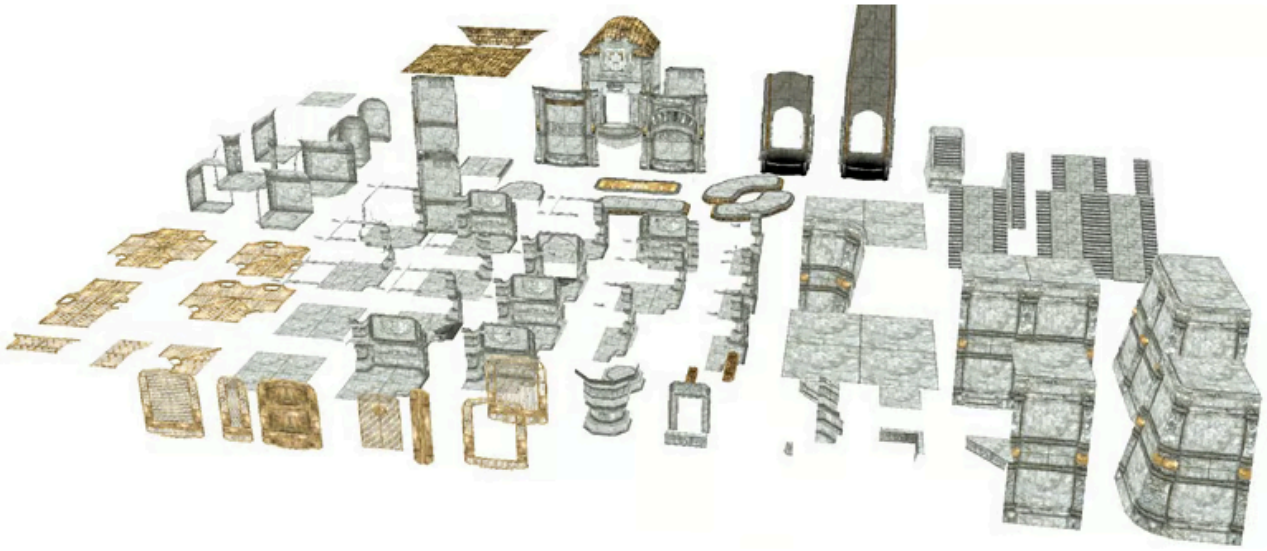
How to measure and design modular kits, 3D tilesets for building levels

When doing a blockout or art pass, you might use a **modular kit** -- a 3D tileset of **modules**, meshes designed to snap together. This comes from the real-life building practice of prefabricated modular construction ↗.

If you're new to 3D modeling or level design, then don't attempt to design your own kit yet. Many considerations go into designing a robust kit. Your first kit will probably be very hard to use, especially if you haven't learned from using other peoples' kits already. Instead, see our list of Resources to download a free pre-made modular kit suitable for prototyping.

But if you have experience in level design and 3D tools, then you are ready. On this page we will detail the many considerations and best practices for designing your own modular kit.

> Most of the info on this page comes from Joel Burgess' excellent GDC talks on designing modular kits for big open world games: "GDC 2013: Skyrim's Modular Approach to Level Design" ↗ (slides ↗) and "GDC 2016: The Modular Level Design of Fallout 4" ↗ (slides ↗, video ↗).



core architectural kit for the "Dwemer ruins" in Skyrim, image by Joel Burgess (http://blog.joelburgess.com/2013/04/skyrims-modular-level-design-gdc-2013.html)



example dungeon environment built with the Dwemer kit in Skyrim, image by Joel Burgess (http://blog.joelburgess.com/2013/04/skyrims-modular-level-design-gdc-2013.html)

1. Pre-production

do you have a lot of level designer time, and need to reuse limited art assets many times?

planning kit percentages, decide what is most important first

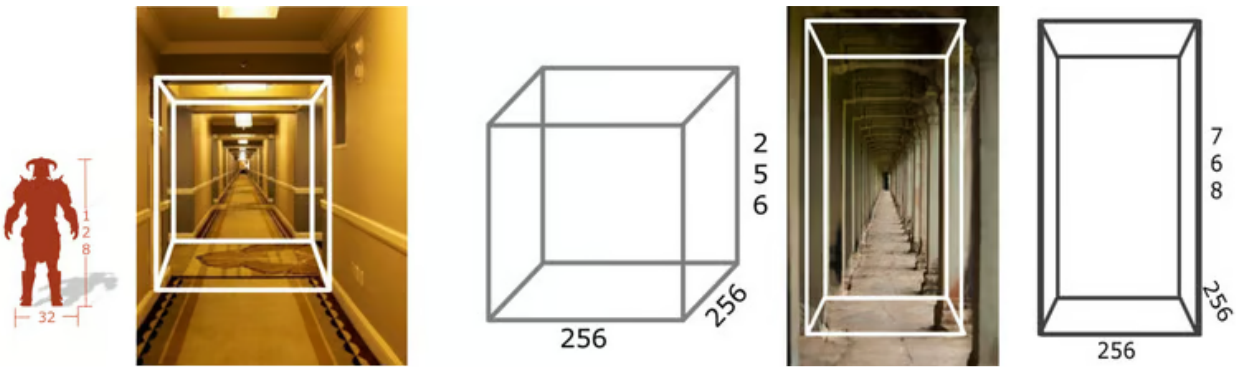
how granular / chunky should the kit be? depends on what level designers need / want, more customization OR faster prototyping time?

2. Blockout the kit

DO NOT MAKE VARIANTS YET, just build basic pieces first

Prototype metrics

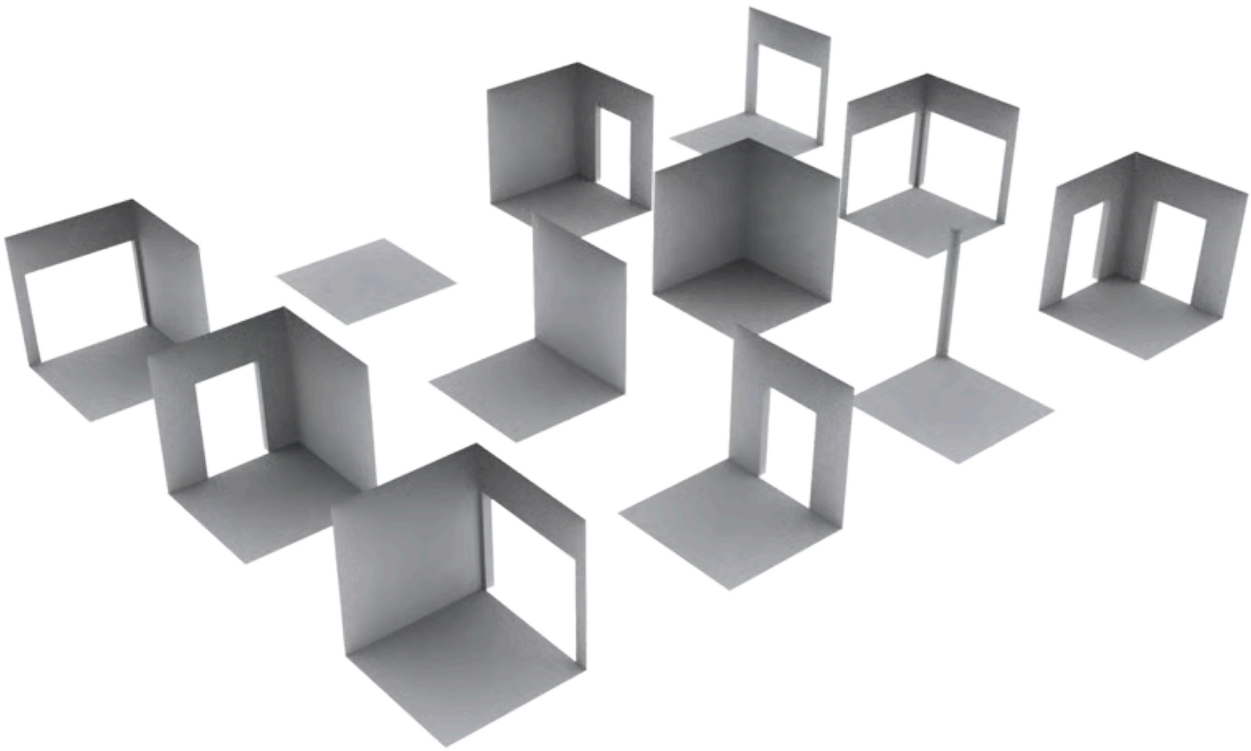
decide on your units... build to a human scale



player size vs environment metrics diagram, image by Joel Burgess (http://blog.joelburgess.com/2013/04/skyrims-modular-level-design-gdc-2013.html)

Common module types

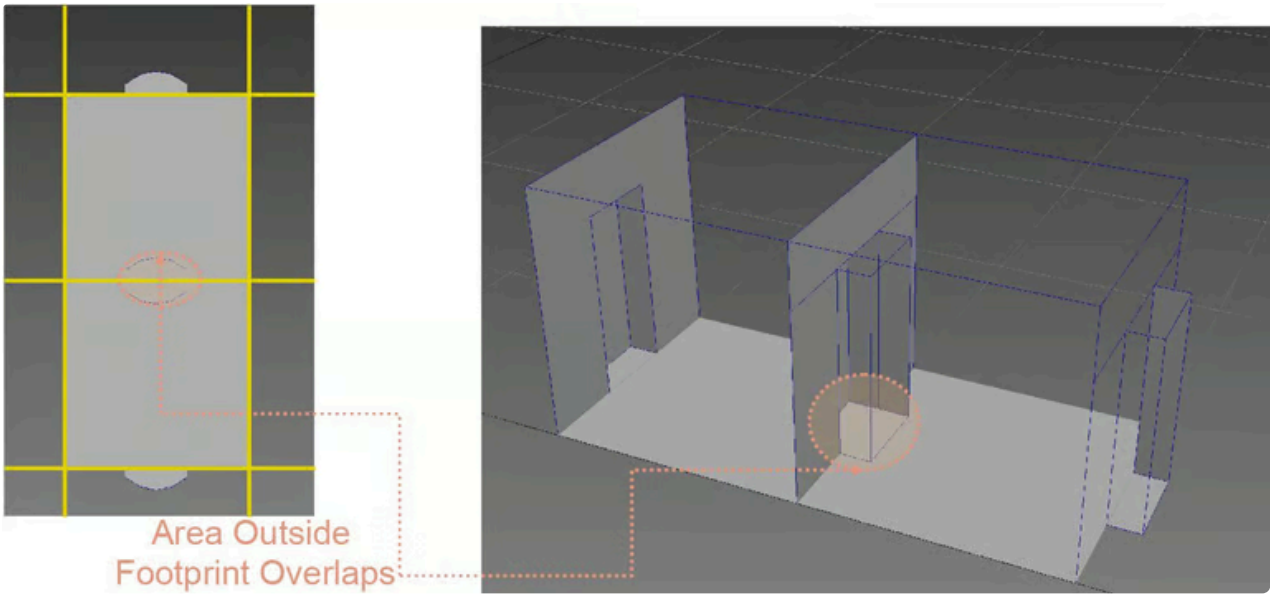
- floor, ceiling, wall, doorway (single), doorway (double), window, corner
- platforms (landscape)
- glue
- flanges and arches (caves)
- shells (caves)



"graybox" modular kit used for level blockouts in Skyrim, image by Joel Burgess (http://blog.joelburgess.com/2013/04/skyrims-modular-level-design-gdc-2013.html)

Define a footprint

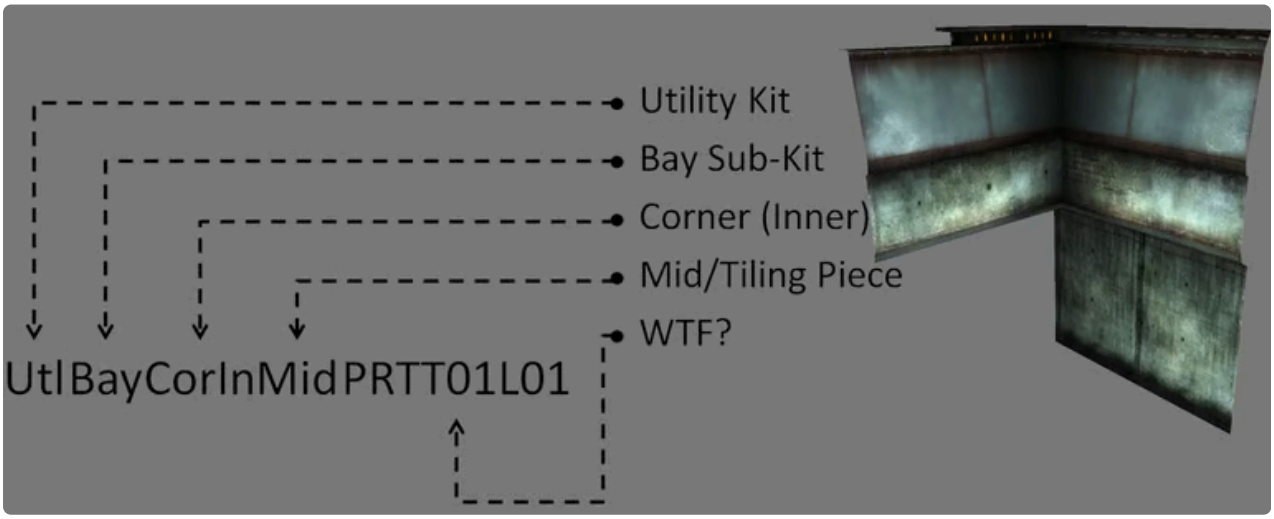
stay in footprint / bounding box



Asset naming

pay attention to naming, level designers and kit artists need to agree on a pattern

Epic's recommended naming scheme?

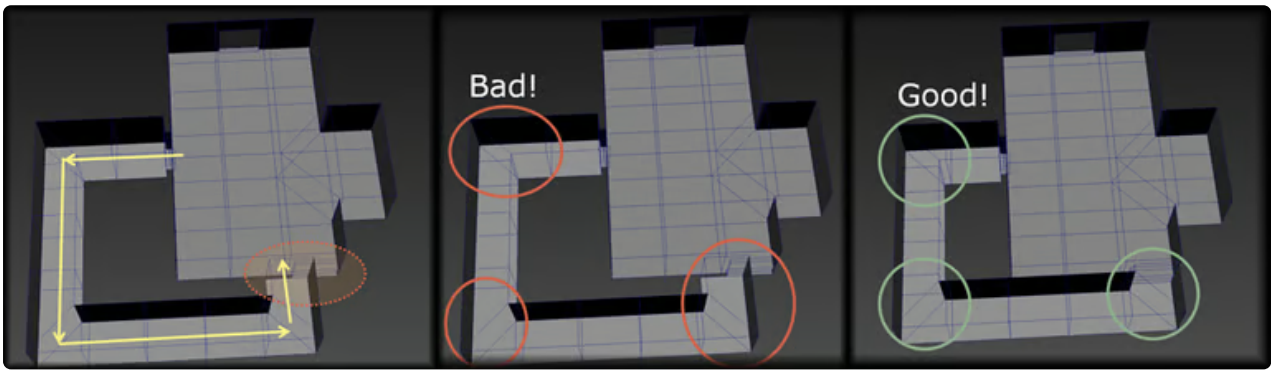


3. Metrics: stress test the kit

do NOT just stop at an ideal case

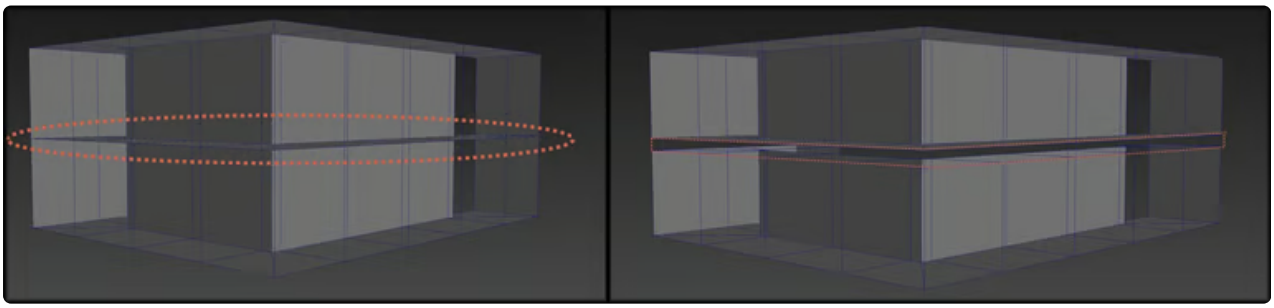
collision is one of the biggest issues, make sure you can't get stuck?

Loopback Test



Stack Test

can you build multi-level environments? floors should NOT be paper thin, they have thickness and mass just like walls



Gap Test

do you have enough glue for interior kits with off-angle construction? essential for tunnel or cave kits

4. Art pass the kit, make variants

texture variations go a long way, and are "cheap" -- you don't have to test and validate the module geometry again

consider building a tool for easily swapping in module types, which will make art pass and remesh much faster

Sources

As mentioned at the top of this page, much of the info here comes from two excellent GDC talks by Joel Burgess, which are a **must-read** for anyone doing modular construction:

- "GDC 2013: Skyrim's Modular Approach to Level Design" ↗ (slides ↗) is the "101" introductory talk to modular kit design considerations, and how they approached building out levels for Fallout 3 and Skyrim, while avoiding the worst of Oblivion's sins.
- "GDC 2016: The Modular Level Design of Fallout 4" ↗ (slides ↗, video ↗) is the "201" intermediate talk that elaborates on what they did for Skyrim's modular kits, and expanded when building Fallout 4.

Further reading

- "Modular Level and Component Design" by Lee Perry ↗ for Game Developer magazine, Nov 2002. A solid article by the lead level designer on Gears of War (2006), one of the first game dev writings to detail modular construction practices -- which only began emerging in the early 2000s when brush-based CSG construction began falling out of fashion. Included here mainly for historical interest; compare against History of the level designer.

Doom metrics

health and damage values, common sizes and dimensions for Doom maps

Below are some core gameplay values and numbers that are useful for level design in Doom / Doom 2.

However, keep in mind this is an action game with aiming and dodging -- so the actual damage and damage per second (DPS) will depend heavily on enemy behavior, available cover, height changes, enemy composition, etc.

For more on what metrics are and why they're useful, see Metrics.



MAP07 "Dead Simple" from Doom 2, by American McGee and Sandy Petersen

World metrics

Units and scale

Doom uses a grid with power-of-two numbers (e.g. 8, 16, 32, 64, 128, 256...) and textures are designed to work in increments of 8, 16, or 32.

Doom's graphics were designed for a 16:10 aspect ratio ↗ stretched vertically by 20% with non-square pixels for a 4:3 display. Today, this results in a lot of its art assets to appear vertically "squished". It also distorts any attempt at a coherent real world scale for Doom, which we can guess at: 16 horizontal Doom units = 10 vertical Doom units = 1 foot = 0.6 meters.

Typical map structure	Width in units
Hallway (very narrow), crate, teleport pad	64
Hallway (narrow), big door, wall textures	128
Small room	256-512
Medium room	512-1024
Large room	1024-1536
Average map size from Doom 1, Episode 1	~4000
Maximum map size (recommended, minimal glitches)	32767 (+/- 16384)
Maximum map size (technical, buggy and unstable)	65535 (+/- 32768)

Maximum map size: to calculate distances, Doom uses 16-bit signed integers which have a maximum value of +/- 32767. However, if you actually built a map that stretched from -32767 to +32767 (across 65535 units!) and somehow tricked the engine into running it, then it would still break other distance calculations like a monster's line of sight, because a value of 65535 would overflow ↗ past +32767 to become 0. For best results, keep all map geometry within 16384 units of the (0, 0, 0) origin.

Weapon metrics

Doom randomly simulates damage values by rolling virtual dice with each hit. For the shotguns, there's an additional buckshot spread simulation where each pellet must connect with the hitbox for full damage.

The damage per second (DPS) is a rough estimate based on the fire rate multiplied by the average damage per shot.

Weapon	Type	Range	Fire rate / sec	Damage / shot	DPS
Fist	Melee	0-32	2 punches	2-20	22
Berserk Fist	Melee	0-32	2 punches	20-200	220
Chainsaw	Melee	0-33	9 revolutions	2-20	90
Pistol	Hitscan	0-512?	2.5 bullets	5-15	25
Shotgun	Near	0-192?	1 shot (7 pellets)	5-15 * 7 = 35-105	70
Super Shotgun	Close	0-128?	1 shot (20 pellets)	5-15 * 20 = 100-300	150
Chaingun	Hitscan	0-512?	9 bullets	5-15	90
Rocket Launcher	Mid / Long	128-512?	1 rocket (+ 128 range splash)	20-160 + 0-128	150
Plasma Gun	Mid	0-384?	12 cells	5-40	270
BFG 9000	Mid	0-384?	1 shot (+ 40 tracers, 1024 range)	100-800 + 49-87	~1200

Most maps begin with players killing low health enemies with the pistol and shotgun. Eventually the player relies more on the chaingun, super shotgun, and rocket launcher, while occasionally switching to the plasma gun for tougher enemies.

The rocket launcher, plasma gun, and BFG are usually less effective at very long range because of the lag built into their projectiles' travel time. At long distances, a monster can move out of the way before getting hit. Doom's autoaiming ↗ and randomized monster movement also means it can be tricky to lead shots. You can balance long range encounters toward the player's favor by placing monsters on pillars with no cover, limiting their ability to dodge the player's projectiles.



Monster metrics

Monster sight and hearing

Monsters can look in cardinal (N, E, S, W) and ordinal (NE, SE, SW, NW) directions, essentially in 45 degree increments. They have a 180 degree sight cone based on their initial facing, and can hear combat sounds based on areas bounded by linedefs ↗ set to block sound.

If set to "ambush" mode, monsters have a 360 degree sight cone and ignore sounds.

Monster movement

Minimum hallway size is given as `(monster width + 2) x (monster height + 2)` but in practice, your hallways should usually be much wider since monsters might "step" in larger increments, and monsters block other monsters. Narrow off-angle hallways will force monsters into slower zig-zag movements, because remember, they can only turn and move in 45 degree increments.

Stairs are tricky for monsters. In general, steps with long depths and shallow rises are always more dependable. Step height must always be 24 units or less (or else the monster won't cross) and minimum step depth / maximum slope is proportional to the monster width. For example, for a step that is 24u high, a trooper requires a step that is 33u deep (35 degree rise, 2:3 ratio) while a demon is wider so it requires 51u deep (25 degrees, 1:2 ratio). If you want to see the bounding box calculations yourself, see the `PCheckPosition()` and `PTryMove()` functions in `p_map.c` ↗ of the Doom source code.

To simplify building for monsters, we generally recommend:

- *Minimum hallway size:* 128 wide x 128 tall, mostly built orthogonally at 90 degree angles to align with the grid with occasional 45 degree angles.
- *Stairway step size*: 16 high x 64 deep (15 degree rise, 1:4 ratio) or 8 high x 32 deep.

Monster combat

Monsters will use melee attacks within 64 units of their target, though the Revenant will attempt to use a melee attack within 196 units even if the target is too far. If further than 64 units, then monsters with ranged attacks are more likely to use attack the closer they are to the player, up to a maximum distance of 2048 units. But the Arch-vile has a particularly dangerous ranged attack, so it will only attack within 896 units.

When hit, monsters have a random chance to be stunned in a pain state ↗-- weapons with fast fire rates (chain gun, plasma gun) or multiple projectiles (shotgun, super shotgun) are particularly good at stunlocking monsters and interrupting their attacks.

For much more on monster behavior and debugging, see "Monster behavior" on The Doom Wiki ↗.

Doom 1 monsters

Monster	Health	Pain chance	Min. hallway size (w x h)	Min. step depth (24u high step)
Player	100	--	33 x 58	1
Zombieman ↗ (trooper)	20	80%	42 x 58	33
Sergeant ↗ (shotgun guy)	30	68%	42 x 58	33
Imp ↗ (fireball demon)	60	80%	42 x 58	33
Lost Soul ↗ (flying skull)	100	100%	34 x 58	--
Demon ↗ / Spectre ↗ (fast pig)	150	71%	62 x 58	51
Cacodemon ↗ (big flyer)	400	50%	64 x 58	--
Baron of Hell ↗ (hunky goat)	1000	17%	50 x 66	41
Spiderdemon ↗ (boss)	3000	13%	258 x 102	254?
Cyberdemon ↗ (final boss)	4000 (+50% rocket resist)	5%	82 x 112	74?

Doom 2 monsters

Doom 2 includes all the Doom 1 monsters, and added more mid tier monsters designed to survive longer and interact with other monsters.

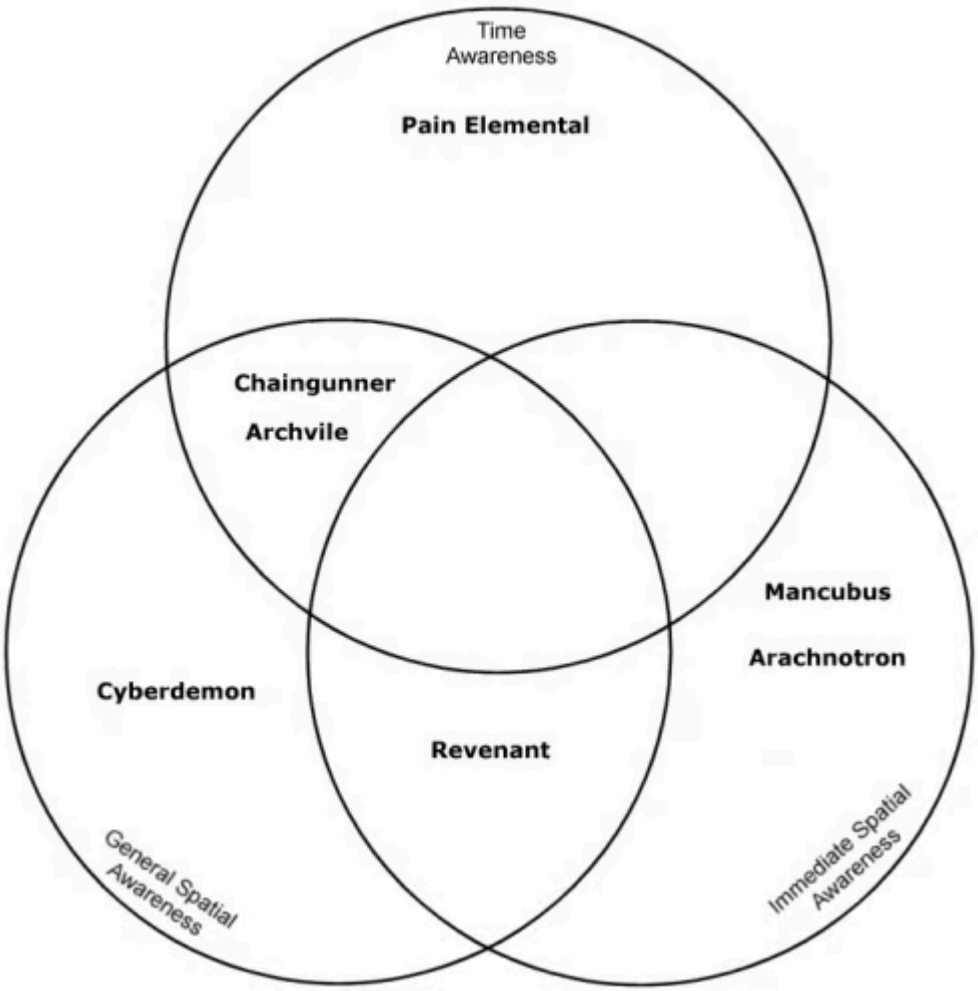
Monster	Health	Pain chance	Min. hallway size (w x h)	Min. step depth(24u high step)
Heavy weapon dude ↗ (chaingunner)	70	68%	42 x 58	33
Revenant ↗ (skeleton with missiles)	300	40%	42 x 58	33
Pain Elemental ↗ (big flyer, spawns Lost Souls)	400	50%	64 x 58	--
Arachnotron ↗ (baby spiderdemon)	500	50%	130 x 66	124?
Hell Knight ↗ (weaker Baron of Hell)	500	17%	50 x 66	41
Mancubus ↗ (big flamethrower monster)	600	31%	98 x 66	90?
Arch-vile ↗ (flame zombie, resurrects monsters)	700	3%	42 x 58	33

Doom monster design analysis

An influential Doomworld poster Linguica offers these helpful design thoughts on Doom monster design:

> *... Doom 2 monsters are great. They nearly all enhance the gameplay along one or more of three axes:*
>
> - ***time awareness*** *- what is happening that I need to immediately address?*
> - ***immediate spatial awareness*** *- what is in my close vicinity right now?*
> - ***general spatial awareness*** *- what is the architectural layout, like walls, buildings, etc, in my area?*
>
> *Doom 1 enemies were designed by what we would consider today to be novice FPS players, for whom basic movement and avoidance were challenging enough, without monsters making it much more difficult. However, as a more advanced FPS player, what tactical problems do you end up having while fighting them? Beyond getting boxed into a dead end, there is practically nothing preventing you from kiting Doom monsters indefinitely with little challenge. The only problems you would run into are being progressively plinked by shotgunners (troopers aren't even worth mentioning) or splash damage from a Cyberdemon's rockets (I'm discounting the Spiderdemon as a "normal" enemy here, but it is basically a giant rapid fire shotgunner in practice anyway, so whatever).*
>
> *Now let's look at the new Doom 2 monsters and what they bring to the table:*
>
> - *The* ***Chaingunner*** *is a "low-level" enemy that even a more advanced player needs to worry about, because especially en masse, they can really wreck you quickly. There is a* ***time awareness*** *factor because if you don't take care of them quickly, your health will be severely eroded. There is also an element of* ***general spatial awareness*** *in case you need to retreat behind cover.*
> - *The* ***Mancubus****'s offset fireballs mean you can't just rely on mindless strafing around projectiles and actually need to pay attention to them, or else you can end up running into one. This implicates your* ***immediate spatial awareness****. (Did you end up learning the "Mancubus dance" of strafing left-right-center? Then congratulations, the game taught you a new behavior.)*

> - *The **Arachnotron**'s rapid-fire plasma also implicates **immediate spatial awareness** because if you're doing any strafing back and forth, you have to watch out for "crossing the stream" and being hit in the process. Do you think it's a coincidence the Mancubus and Arachnotron are introduced in quick succession on the same level, which also includes no other enemies? I don't think so. Both enemies make you do more than the Doom 1 hallmark of simple strafing around a single fireball. The game is telling you, in case you haven't gotten the point yet, that this is not going to work anymore.*
> - *The **Pain Elemental** is a annoying enemy, yes, but it is there purely to implicate your **time awareness** - it can't even hurt you directly! Its danger to you is directly proportional to how long you let it stick around.*
> - *The **Revenant**'s homing missiles, again, mean you can't just rely on mindless strafing around projectiles and actually need to pay attention to them, which involves your immediate spatial awareness, and you also need to keep mindful of available cover so you can get rid of the homing fireball, which involves **general spatial awareness**. You could even argue it involves your **time awareness**, since if you let a large number of homing fireballs accumulate, they could end up killing you instantly. (Or, perhaps, another enemy??)*
> - *The **Archvile**'s fire attack strongly implicates your **general spatial awareness** and **time awareness** - you need to know where cover is, and you need to get there immediately. Furthermore, its secondary behavior of resurrecting dead enemies involves yet more **time awareness** - if you leave it alone too long, it's going to being back all those enemies you already went to the effort to kill.*
> - *The **Hell Knight** is the only new enemy that doesn't make the game tactically deeper along one of these axes. It's just a Baron of Hell without being quite so tedious to fight as a real Baron. This does give mappers a new option for when and how to use the Baron, though, which is a good thing.*
>
> -- Linguica on 28 August 2014 in thread "Doom 1 or Doom 2?" on Doomworld ↗



venn diagram comparing "awareness" metrics for various Doom 2 monsters; by Linguica (via Doomworld ↗)

Item Economy

Ammo pickups

	Small	Large	Default ammo	Monster drop
Bullets	Clip ↗ (10)	Box of bullets ↗ (50)	Pistol ↗, chaingun ↗ (20)	Zombieman (5, or 10 on Hard+)
Shells	4 shotgun shells ↗ (4)	Box of shotgun shells ↗ (20)	Shotgun ↗ (8), super shotgun ↗ (8) (in Doom II ↗)	
Rockets	Rocket ↗ (1)	Box of rockets ↗ (5)	Rocket launcher ↗ (2)	
Cells	Energy cell ↗ (20)	Energy cell pack ↗ (100)	Plasma rifle ↗ (40), BFG9000 ↗ (40)	

Sources

- This page uses data from the Doom Wiki ↗, under CC-BY-SA 4.0 International license.
- Map unit - The Doom Wiki ↗
- Doom Metrics ↗ by Scott Ampoker

Quake metrics

player size and speed, health and damage values, common sizes and building dimensions for designing single player Quake 1 maps

Below are some core gameplay values and numbers that are useful for level design in Quake 1.

Sometimes popular mods evolve new design norms in the community. After all, it's been more than 25 years since Quake's 1996 release. We note differences against default retail unmodded Quake ("Vanilla Quake").

> *For more on what metrics are and why they're useful, see Metrics.*

> *For lots of Quake-specific info and links, see Quake resources.*



screenshot of Quake 1 by id Software, kind of like how it appeared back in 1996

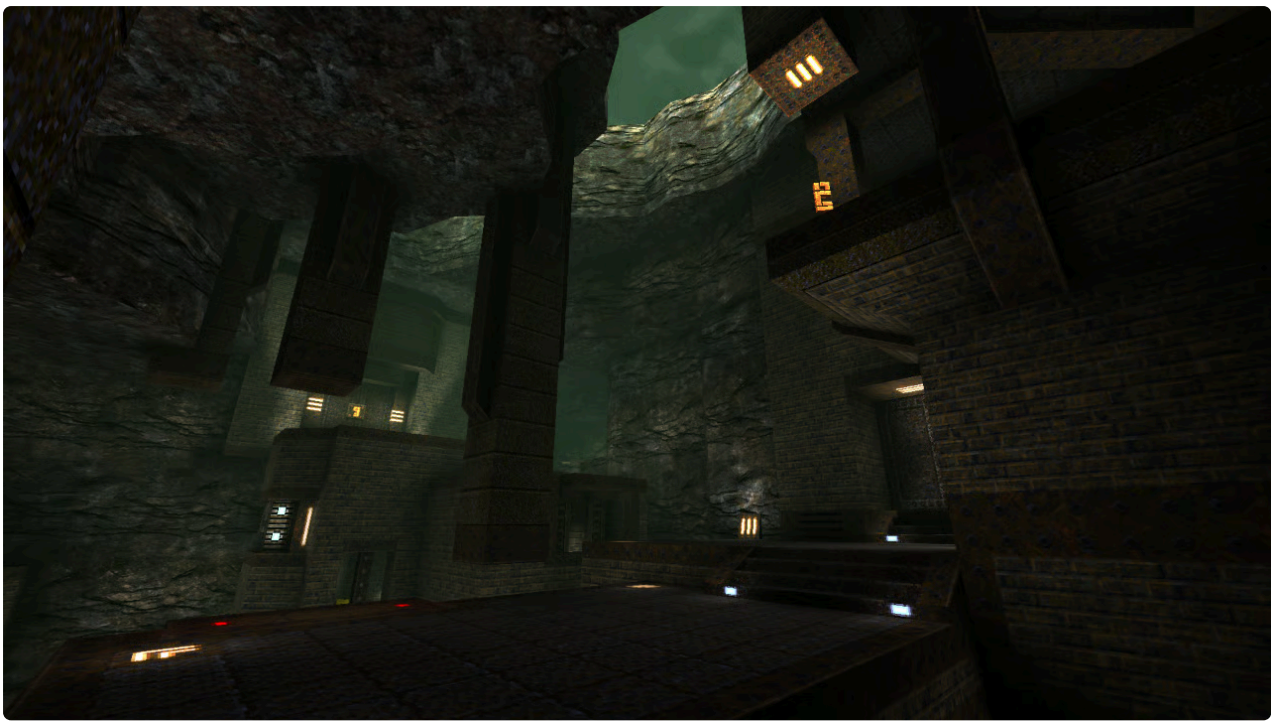
Core player metrics

Quake uses the Quake Engine, obviously. So any basic scaling and metrics recommendations from the main Metrics page about Quake-lineage engines (Source Engine, etc) also apply here.

- 1 Quake unit (u) = 1 inch, but not really. There is no consistent scale.
- Z-axis up
- Quake uses a "power-of-two" grid. Most mappers blockout at a grid size of 64 or 32, then shift down to grid size 16 or 8 to start refining details. Use grid 4, 2, or 1 only for very small details.
- There's no crouching, sprinting, or +use key / interact button.

Player metric	Measurements (L x W x H)
Collision hull size	32 x 32 x 56
Absolute minimum hallway width	33
Absolute minimum ceiling height	57
ViewPos / camera height above floor	46 (24u mins extent + 22u view offset)
Maximum run speed	320 units / second
Fall damage height*	256+
Absolute maximum step height**	18
Maximum climbable slope**	45 degree incline?
Underwater height	29+ (can die via Thunderbolt discharge)
Max jump distance (running start)***	244
Max standing jump height***	43
Ramp jump ↗***	96 high?
Grenade jump ↗***	~320 long - ~128 high?
Rocket jump ↗***	~320 long - ~128 high?

- *Falling damage is -5 health if the player hits the ground at any downward vertical speed faster than 650 units / second. In practice, this means fall damage rarely kills players. To kill the player from a tall drop, create a `trigger_hurt` or `trigger_void` (implemented in some mods).
    - Consider alternate ways of punishing the player for falling (monsters below, or water, slime, lava) or maybe even just let the player fall down, explore, and find their way back up again.
- ** Step height and slope assumes orthogonal (90 degree) axis-aligned surfaces. Quake collision uses Axis-Aligned Bounding Boxes (AABB) with no real support for rotations, so walking up a slope along a diagonal will feel much steeper.
    - Best practice is for ramps and slopes to be axis-aligned.
    - Many maps use Doom-style stepped terrain instead of smooth sloped terrain, partly to prevent physics problems. Monsters seem to like it better too.
- *** Jump distances and heights assume the absolute limits of a typical player.
    - Build for a shorter jump distance / height. Most players don't have perfect reflexes and timing.
    - Don't force players to ramp jump, grenade jump, or rocket jump on the critical path; reserve these specialty jumps for optional shortcuts or secrets.
    - Skilled players can use bunnyhopping↗ to build-up more speed and distance, change direction in mid-air, etc. But don't worry about it. Let these players break your map, it's OK.



screenshot of a level from "Underdark Overbright" (UDOB) ↗ by Matthew 'Lunaran' Breit

Common Quake building metrics

Building element	Recommended / typical dimensions
Ceiling, low	64u high, bare minimum; bigger monsters won't fit
Ceiling, regular	128u high; recommended minimum
Ceiling, tall	256u+ high
Railing / tabletop	32u high
Ventilation duct	64u wide, 64u tall (there's no crouch button)
Hallway, regular	128u+ wide
Room, small	256u wide
Room, medium	512u wide
Room, large	1024u+ wide
Ramp	1:2 slope gradient (26.57° incline)
Stairs, simple	16 rise : 32 run (1:2 slope)
Stairs, dense	8 rise : 16 run (1:2 slope)
Gravity	800 units / second^2
Low gravity	100 units / second^2 (in E1M8 "Ziggurat Vertigo")

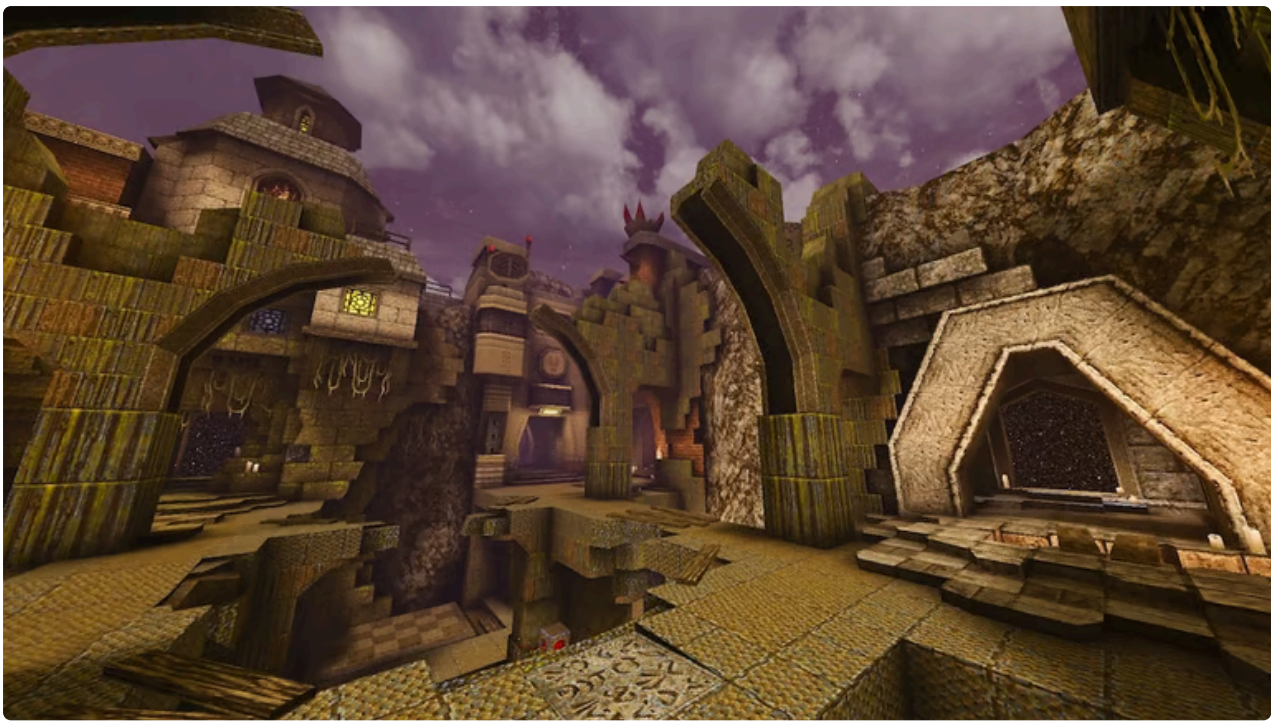
Player weapon metrics

Keep in mind this is an action game with aiming and dodging. The actual damage depends heavily on enemy behavior, available cover, height changes, enemy composition, etc. "Damage per second" is just a rough estimate.

Weapon	Max ammo	Ammo/sec	Damage	DPS	Range	Spr diu
Axe ↗	--	--	20	40	64u	--
Shotgun ↗*	100 Shells	-2.0	24 (4x6)	48	2048u	+/-
Super Shotgun ↗*	100 Shells	-2.8	56 (4x14)	80	2048u	+/-
Nailgun ↗	200 Nails	-10.0	9	90	6000u	--
Super Nailgun ↗	200 Nails	-10.0	18	180	6000u	--
Grenade Launcher ↗	100 Rockets	-1.66	40-120	200	~900u	160
Rocket Launcher ↗	100 Rockets	-1.2	40-120	150	5000u	160
Thunderbolt ↗**	100 Cells	-10.0	30	300	600u	--

- *Shotguns are a little complicated:
  - Damage is calculated via hitscan spread; for full damage, all hitscans must hit the target.
  - The *effective range* is much lower than 2048u.
  - **Experienced Quake players use the default Shotgun more like a pistol** since it has narrow spread, faster fire, and low ammo cost.
- ** Firing the Thunderbolt underwater causes an explosion that kills the player. This "discharge" consumes all Cells and inflicts `cells x 35` damage. With 100 cells, that inflicts 3500 damage on *anything that can see the player underwater*... including the player.
  - but if the player has the invincibility powerup, they will survive.

In vanilla Quake, crowd control in a narrow hallway is harder than it seems -- weapons do not penetrate monsters in death animations and explosive splash damage is buggy. Recent mods like Copper ↗ or Arcane Dimensions ↗ fix these bugs.



screenshot of hub level from Arcane Dimensions ↗, by Simon 'sock' O'Callaghan et al

Monster metrics

Quake Monster	Health	Damage	Hull (WxH)	Hitbox (WxH)
Rotfish ↗	25	3-6 (Bite)	32 x 56	32 x 48
Rottweiler ↗ (dog)	25	1-24 (Bite), 10-20 (Jump)	64 x 88	64 x 64
Grunt ↗ (shotgun)	30	1-16 (Hitscan Shotgun)	32 x 56	32 x 64
Zombie ↗	60 + revives until gibbed	10 (Flesh Shot)	32 x 56	32 x 64
Knight ↗	75	12-18 (Sword), 20-30 (Charge)	32 x 56	32 x 64
Enforcer ↗ (laser)	80	15 (Laser Shot)	32 x 56	32 x 64
Scrag ↗ (flyer)	80	9 (Projectile)	32 x 56	32 x 64
Spawn ↗ (blob)	80	10-20 (Jump), 120 (Explosion)	32 x 56	32 x 64
Ogre ↗ (big dude)	200	4-12 (Chainsaw), 40 (Grenade)	64 x 88	64 x 88
Death Knight ↗	250	20-36 (Sword), 9 x 6 (Bolts)	32 x 56	32 x 64
Fiend ↗ (jumper)	300	40-50 (Jump), 10-15 (Claw)	64 x 88	64 x 88
Vore ↗ (spider legs)	400	40 ("Voreball" Homing Shot)	64 x 88	64 x 72
Shambler ↗ (big furry)	600 + 50% explosive resist	10-30 (Lightning, range: 600), 80-120 (Claw)	64 x 88	64 x 88

Monster collision

Quake has two collision systems. QBSP bakes **collision hulls** for basic movement clipping (entity vs world collisions), and then there's also **hitboxes** for moving objects (entity vs entity collisions). Both collision types use AABB (axis-aligned bounding box) shapes that cannot rotate.

There are three (3) world collision hull sizes:

- **hull 0: 1x1x1** ("points" like projectiles, missiles)
  - but a missile's *hitbox* is 32x32x32 (via movetype `FLYMISSILE` / `BOUNCEMISSILE`)
  - and in-game, missiles don't even use this hull, and traverse the BSP instead
- **hull 1: 32x32x56** ("small" players, humanoids)
- **hull 2: 64x64x88** (large monsters)

Meanwhile, hitbox sizes vary slightly for every monster, and are listed in the table above.

ADVICE: HOW TO AVOID STUCK MONSTERS

Allow more extra space beyond the monster's size or else it will have trouble moving around: **at least 16 units above ground / 16 units away from walls.** Placing monsters slightly in the air is common.

Generally, place monsters **in spaces at least *double* their width and height**.

- For example, an Ogre (64 x 88) is better in a room that's 128+ wide and 176+ tall. For ease of construction, it's common to round up the room height to an increment of 64 (192, 256).
- Fiends and Spawns jump a lot and definitely need a 128+ tall room, ideally 256.
- But sometimes less monster movement is better, it all depends on the level.

Gibbing

Gibbing occurs when monsters get damaged a lot, all at once. If their health drops below a certain negative threshold quickly enough, then they turn into "gib" pieces instead of playing their death animation. This generally only happens when the player uses explosive weapons (grenade launcher, rocket launcher) or the quad damage powerup.

In vanilla Quake this mechanic only matters for zombies, who revive themselves unless they lose all their health (60) within a single frame and become gibbed. Again, the player will usually need explosives or a quad to kill the zombies permanently.

Infighting

Infighting happens when a monster damages a *different* monster type, which causes the hurt monster to target the other monster instead of the player.

For example, when an ogre's grenade accidentally hurts a fiend, the fiend will turn around and attack the ogre. But if an enforcer's laser hits another enforcer, no infighting will occur.

Melee-only monsters will never cause infighting. Only ranged monsters can.

LESS INFIGHTING	MORE INFIGHTING
Just melee or just ranged types	Mix melee and ranged types
Fewer monsters, fewer types	More monsters, more types
Elevate / leash ranged monsters	Keep ranged monsters at ground level
Spread out monsters	Pack monsters together tightly
Wide open space	Smaller space with chokepoints

Frequent infighting can make encounters feel too easy and unbalanced. But sometimes infighting makes for a more interesting battle. It all depends.

It is possible to design your map to *require* infighting by placing many monsters / less ammo. However, balancing these types of combat puzzles is complicated. Make sure to set a clear player expectation that infighting is required, otherwise players will be confused why there wasn't enough ammo. We recommend designing these situations only for Hard mode.

Skill level / difficulty levels

Quake has 4 difficulty levels: `skill 0` "Easy", `skill 1` "Normal", `skill 2` "Hard", and `skill 3` "Nightmare".

In general, health and damage do not scale automatically with difficulty level; skills 0-2 depend on mappers to manually adjust monster count and items.

The exception is skill 3 Nightmare, which enforces several code-level changes:

- 1996 vanilla Nightmare:
  - faster monster attack speed / frequency
  - less likely to stun monsters (enter "pain frame" animations)
  - Shambler lightning lasts 33% longer, for a maximum of 40 damage
  - Vore balls move 40% faster
- 2021 remaster / Copper Nightmare↗:
  - all 1996 Nightmare-specific changes reverted; Hard is now identical to Nightmare
  - except the player's maximum health is now 50 instead of 100

Item metrics

Ammo

There are three ways to get ammo: pickup an ammo **item**, pickup a **weapon**, or pickup a **drop** from a dead monster.

Ammo pickup / drop	Source	Type	Amount
item_shells	**Item**	Shells	**+20 (large: +40)**
weapon_shotgun	Weapon	Shells	+25
weapon_supershotgun	Weapon	Shells	+5
monster_grunt	Drop	Shells	+5
item_spikes	**Item**	Nails	**+25 (large: +50)**
weapon_nailgun	Weapon	Nails	+30
weapon_supernailgun	Weapon	Nails	+30
item_rockets	**Item**	Rockets	**+5 (large: +10)**
weapon_grenadelauncher	Weapon	Rockets	+5
weapon_rocketlauncher	Weapon	Rockets	+5
monster_ogre	Drop	Rockets	+2
item_cells	**Item**	Cells	**+6 (large: +12)**
weapon_lightning	Weapon	Cells	+15
monster_enforcer	Drop	Cells	+5

- All ammo item pickups can be flagged as "large", which doubles the ammo they provide.
- Gibbed↗ **monsters do not drop ammo.**
- Army grunts die to 2 shotgun shots, but drop 5 shells, resulting in +3 "profit". If you use a lot of these monsters in the level, then balance your item_shell quantity accordingly.

- Ogres die to two rockets, but also drop two rockets, so using rockets on them is "free." Or alternatively, you can think of them as a way to exchange other ammo types for rockets.
  - For this reason, the rebalance mod Copper reduces the ogre drop to only 1 rocket.
  - But don't *require* players to use any of this "economic thinking." See Advice.
- Nails / spikes are the only ammo type that do not drop from monsters.

Armor

Player armor follows an overcomplicated formula, and honestly, we recommend just ignoring these details. It's not important. But for the sake of completeness:

- When the player gets hurt while wearing armor, they lose armor points (instead of health) based on the armor type's damage absorption rate.
  - *example: Red Armor will absorb 80% of the damage you receive and subtract it from your armor points instead -- in practice, that means Red Armor rarely lasts very long, even though it seems like you get more of it.*
- If you have too much of a better armor color, then you can't pick up a worse armor color.
  - *example: if you have 114+ points of Red Armor, then you can't pickup Yellow Armor.*

Armor type	Armor amount	Absorption	Armor switch threshold
Green Armor ↗ *(item_armor1)*	+100	~33%	--
Yellow Armor ↗ *(item_armor2)*	+150	~60%	50 or lower (to Green)
Red Armor ↗ *(item_armorInv)*	+200	~80%	113 or lower (to Yellow) 38 or lower (to Green)

- Green is common in single player maps
- Yellow is considered to be pretty powerful, use carefully and sparingly
- Traditionally, Red is very rare, and reserved more as a reward for a secret

Powerups

Powerup type	Effect	Duration
Biosuit ↗	Breathe underwater, zero slime damage, lava does less damage	30 seconds
Ring of Shadows ↗	Invisibility; but once you attack a monster, it sees you	30 seconds
Megahealth ↗	+100 health, ignores max health; stack up to 250	-1 Health > 100 every 5 sec.
Pentagram ↗	Invulnerability, immune to telefrag and thunderbolt water discharge	30 seconds
Quad Damage ↗	All damage multiplied by 4; more likely to gib monsters	30 seconds

- in vanilla Quake, the Biosuit and Ring are considered lackluster powerups with limited benefit that depend heavily on the level designer
- the Quad is probably the most interesting powerup
  - lets the player gib zombies with just the Axe or Shotgun
  - gibs enemies more often, which means they don't drop ammo
  - but also an opportunity to conserve ammo, e.g. use 1 rocket to clear an entire room
  - splash damage from grenades / rockets can easily backfire, killing the player instantly

Advice

- Quake monsters aren't very clever, which is part of the charm. Think of their movement more like semi-chaotic pinball rather than a tactical cover shooter.
- Give more ammo than the player will need. A first playthrough will waste ammo. If the player theoretically needs only 20 shotgun shells used with perfect economy and accuracy, then you should probably give at least 40.

For more general advice on designing combat, see Encounters.



example of increasingly challenging shotgun placements by Lunaran from "Balancing Skill Levels for Quake" ↗

Lastly, here's veteran Quake modder Matt "Lunaran" Breit with some Quake-specific advice and approaches for balancing encounters, item placement, and difficulty ↗:

> I think resisting "easier is just fewer monsters" design is the right way to go. There should be fewer monsters, just not many fewer, because too few is just plain boring. We're all Quake Experts after 20 years of this, so I think anyone playing custom maps on Easy in 2018 is doing so because they're doing it on a lunch break or a stolen evening away from the kids, and not because they can't handle more than one fiend at a time. Maybe we should think of it more as 'higher investment.' Besides, Quake gives you tons of unappreciated variables that you can tweak by skill, both obvious and subtle.
>
> - Armor makes Quake significantly easier. More reds and yellows, more often, effectively extends the player's survivability in a given fight by hundreds of HP. More Greens, or stretches without armor at all, shrink the until-death buffer to little more than the player's current health. If you do want to keep the same monster loadout on all three skills, give the Easy player several Red and Yellow armors and the Hard player only one or two Greens. They'll feel like completely different games.
> - Adding a vote for weapon pickups coming earlier or later. On Easy, the next big weapon might come before the next big encounter so the player can kick ass with it, on Medium it might be placed within it so the player has to engage to grab it, and on Hard it might only come as a reward after beating the fight entirely without it.
> - Greater monster variety leads to more ways the player can be attacked at any one time, requiring juggling more variables to avoid damage and find the safe place to be standing at any given millisecond. A shambler and a vore together are harder to handle than a pair of either. Variety also raises the chances of infighting, however.
> - The angles that enemies are presented from makes a difference. In front of the player is easier, flanking is harder, behind is bordering on unfair depending on circumstances. Below the player is a turkey shoot, eye level is straightforward, and monsters up high have a distinct advantage.
> - Quantity of resources matters, of course. Bigger medkit pools clearly make the game easier, plentiful rockets can be splashed around while rare ones are only for emergencies, etc. Nail weapon DPS is higher than SG/SSG DPS and so on.

- Frequency of resources matters too. A steady drip lets the player feel secure, but isolated bursts create situations where the player has to stretch himself to get to the next 'island'. Depending on where he makes his errors, he might have to stretch pretty hard (eg those 'quicksave with 5 health left' or 'shambler axe dance or bust' moments). Feast-or-famine item placement can induce mild stockholm syndrome, leading to more positive reviews :)
- Unless you're using a lot of Enforcers, maybe provide all players an early Lightning Gun and simply vary the cells provided, as a way of dealing more or fewer 'get out of jail free' cards.
- Don't forget that the difficulty spawnflags are present on every entity. If you're using monster closets, vary the locations of the ambush triggers. Have the Hard ambushes happen when the player is in the worst possible position, and give them a leg up or more warning on easier skills (or even leave the closet open on Easy so there's no surprise at all). You might even duplicate the doors so you can set different 'speed' keys per skill, so the harder ambushes are an instant pants shitting and the easier ones are more like a countdown until the monsters come out, complete with early warning aggro sounds. Doors can be temporarily barred behind the player on hard skills while he is free to retreat from a fight on easier ones. Falling into a pit can be a mild backtracking inconvenience on easy skills but death by spikes on harder ones. How much room is there between nail shooters in this hallway? With careful use of triggerable lights and skill-specific trigger_relays, you can even use light and darkness against the player differently.

Getting crafty with what you change between difficulty levels can give you ideas for entire encounters, although don't rely on that too much for interest, because any given player is probably only going to experience one such permutation and thus won't realize the need to appreciate how different it is from any others.

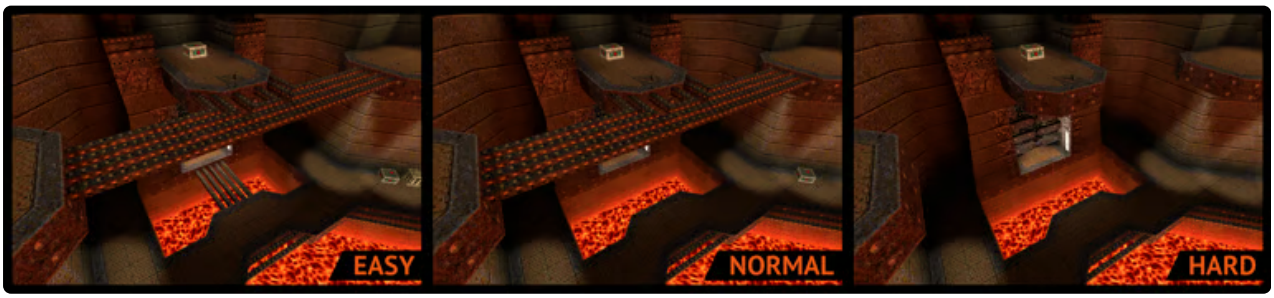
Here's a method I've been using. It's really rough, and time consuming without a custom progs to do it for you, but it can be a helpful way to ground your estimates.

A box of: 25 or 50 nails = 225 or 450 damage 20 or 40 shells = 440/880 dmg 6 or 12 cells = 180/360 dmg

6/12 rockets are harder to judge because of 1) splash damage and 2) zombies, but let's say every rocket is 180dmg, for 1080/2160 dmg per box.

> Total all the ammo you provide in the map (add 25 shells for the starter ammo, add 2 rockets per ogre and 5 shells per grunt and so on), and that's the max amount of hit point damage you are giving the player to deal. Total the starting health of all the monsters, and compare the two numbers.
>
> Researching id maps and popular custom maps reveals an average 'custom' of about ~3:1 on Easy, ~2.2.:1 on Medium, and ~1.7:1 on Hard. the id maps are generally above that curve (4/3/2:1), and custom maps tend to fall below it(2.5/2.0/1.5:1).
>
> Careful cheapskate shot-counters can finish a map with a ratio of about 1.3:1 ammo DP:monster HP, but most players will have to resort to the axe at some point and will complain of shortage. RPGSP1, which was greeted by universal reviews of "good but I ran out of ammo at the end" still had a ratio on skill 2 of 1.4:1 DP:HP.
>
> There are lots of outliers to these curves, though, because so much of it comes down to how the level design enables the player to use the weapons, as well as exploit infighting, choke points, etc. Do rockets get spent one at a time on individual zombies or can they be used to gib crowds of knights for maximum ROI? or are they useless against herds of shamblers?
>
> It also matters when the player gets the ammo. Ammo the player doesn't pick up or can't use is effectively not present in the map at all. Does it come too late to be used when it was really needed? does it come too early and get skipped? or partially wasted when picked up by a player who's already nearly maxed and getting too much at the wrong times? How much of that ammo is in secret areas?

Also keep in mind, you can toggle different platforms / walkways per difficulty, to make traversal easier or harder for the player. It's not just about item and monsters.



examples of Easy / Normal / Hard traversals, by Lunaran via "Balancing Skill Levels for Quake" ↗

Further reading

- Copper - Changes ↗ is an epic 5000 word design essay on Quake's design and how modder Matt "Lunaran" Breit approached his influential rebalance mod Copper.
- We have a dedicated Quake resources page with a lot more info and links.

Sources

- QuakeWiki.org ↗ - see list of Monsters ↗, list of Weapons ↗, under GNU Free Documentation License v1.3+ ↗
- Quake Wiki (Fandom) ↗, under Creative Commons CC BY-SA 3.0 ↗ license